

Software Engineering

Software engineering is a branch of computer science that deals with the design, development, testing, and maintenance of software applications. Software engineers apply engineering principles and knowledge of programming languages to build software solutions for end users.

Some of the topics that software engineering covers are:

- **Software development life cycle:** The process of planning, creating, testing, deploying, and maintaining software products. There are different models of software development life cycle, such as waterfall, agile, spiral, and iterative.
- **Software requirements:** The specifications of what the software should do, how it should behave, and what constraints it should meet. Software requirements can be functional, non-functional, or quality attributes.
- **Software design:** The process of defining the architecture, components, interfaces, and data structures of the software system. Software design can be done using different methods, such as object-oriented, structured, or functional.
- **Software testing:** The process of verifying and validating that the software meets the requirements and expectations of the stakeholders. Software testing can be done at different levels, such as unit, integration, system, and acceptance.
- **Software maintenance:** The process of modifying and improving the software after it is deployed. Software maintenance can be done for different reasons, such as correcting errors, enhancing features, or adapting to changes in the environment.
- **Software quality:** The degree to which the software satisfies the requirements, expectations, and standards of the stakeholders. Software quality can be measured using different criteria, such as reliability, usability, efficiency, maintainability, and portability.
- **Software engineering ethics:** The principles and values that guide the professional conduct of software engineers. Software engineering ethics can be based on codes of ethics, such as the ACM Code of Ethics and Professional Conduct or the IEEE Code of Ethics.

Software engineering is a dynamic and evolving field that requires continuous learning and adaptation. Software engineers need to have strong analytical, problem-solving, and communication skills, as well as a passion for technology and innovation.

Unit 1 - Introduction to Software Engineering

- Software engineering is the application of engineering principles and practices to the development, maintenance, and evolution of software systems that meet the needs and expectations of stakeholders.

- Software engineering involves activities such as:
 - Requirements analysis: eliciting, specifying, and validating the functional and non-functional requirements of the software system.
 - Design: creating a high-level and low-level architecture of the software system that satisfies the requirements and constraints.
 - Implementation: writing, testing, and debugging the source code of the software system according to the design.
 - Verification and validation: ensuring that the software system meets the requirements and quality standards, and performs as expected.
 - Deployment: delivering, installing, and configuring the software system for the end-users or customers.
 - Maintenance: fixing errors, improving performance, adding features, and adapting the software system to changing needs and environments.
 - Evolution: managing the changes and updates to the software system over its lifetime.
- Software engineering also involves processes, methods, tools, standards, and best practices that support and guide the software development activities and ensure their quality and efficiency.
- Software engineering is a multidisciplinary field that requires knowledge and skills from various domains, such as computer science, mathematics, engineering, management, communication, and domain-specific expertise.
- Software engineering is a dynamic and evolving field that responds to the challenges and opportunities of new technologies, paradigms, and applications of software systems.

Introduction to Software Engineering

- Software engineering is the application of engineering principles and practices to the design, development, testing, deployment, and maintenance of software systems.
- Software engineering aims to produce high-quality software that meets the needs and expectations of the users, clients, and stakeholders, while adhering to the budget, schedule, and constraints of the project.
- Software engineering involves various activities, such as:
 - Requirements analysis: eliciting, specifying, and validating the functional and non-functional requirements of the software system.
 - Design: creating the architecture, components, interfaces, data structures, algorithms, and protocols of the software system.
 - Implementation: coding, debugging, and testing the software system using programming languages, tools, and frameworks.
 - Verification and validation: ensuring that the software system conforms to the requirements and specifications, and meets the quality standards and criteria.

- Deployment: installing, configuring, and launching the software system in the target environment, such as a server, a device, or a cloud platform.
- Maintenance: providing support, updates, enhancements, and fixes for the software system throughout its lifecycle.
- Evolution: adapting, modifying, and improving the software system to cope with changing requirements, technologies, and user feedback.
- Software engineering also involves various processes, methods, models, standards, and best practices, such as:
 - Software development lifecycle (SDLC): a framework that defines the phases, activities, deliverables, and roles involved in a software project, such as waterfall, agile, or spiral models.
 - Software development methodologies: a set of principles, practices, and techniques that guide and facilitate the software development process, such as scrum, kanban, or extreme programming.
 - Software engineering ethics: a set of values, principles, and codes of conduct that regulate the professional behavior and responsibilities of software engineers, such as the ACM/IEEE Software Engineering Code of Ethics and Professional Practice.
 - Software engineering standards: a set of rules, guidelines, and specifications that define the quality, performance, and compatibility of software products and processes, such as the ISO/IEC 25000 series on software quality requirements and evaluation.
 - Software engineering tools: a set of software applications, libraries, frameworks, and platforms that support and automate the software engineering activities, such as integrated development environments (IDEs), code editors, compilers, debuggers, testing tools, configuration management tools, and deployment tools.

Software Components

- Software components are parts of a system or application that perform a specific function and can be reused in different contexts.
- Software components have well-defined interfaces that specify the inputs, outputs, and behavior of the component.
- Software components can be deployed independently and composed by third parties to create larger systems.
- Software components can be classified into different types, such as:
 - Application components: These are components that provide specific functionality for an application, such as a user interface, a database, or a business logic module.
 - Infrastructure components: These are components that provide common services and utilities for other components, such as logging, security, or communication.

- Middleware components: These are components that facilitate the interaction and integration of other components, such as message brokers, web servers, or application servers.
- Framework components: These are components that provide a generic structure and behavior for a domain or a technology, such as a web framework, a game engine, or a testing framework.
- Software components are developed and managed using component-based software engineering (CBSE), which is a branch of software engineering that emphasizes the separation of concerns, the reuse of existing components, and the modularity of software systems.

Software Characteristics

Software characteristics are the attributes or qualities that describe a software product. They are also known as software quality factors or software quality attributes. Software characteristics affect how the software performs, how easy it is to use, maintain, and modify, and how reliable and secure it is. Some of the common software characteristics are:

- **Functionality:** The degree to which the software meets the specified requirements and provides the desired functions. Functionality can be measured by criteria such as correctness, completeness, interoperability, compliance, and security.
- **Reliability:** The degree to which the software performs consistently and accurately under normal and abnormal conditions. Reliability can be measured by criteria such as availability, fault tolerance, recoverability, and maturity.
- **Usability:** The degree to which the software is easy to learn, operate, and understand by the intended users. Usability can be measured by criteria such as learnability, efficiency, effectiveness, satisfaction, and accessibility.
- **Efficiency:** The degree to which the software uses the available resources optimally and minimizes the waste of time, space, and energy. Efficiency can be measured by criteria such as time behavior, resource utilization, and capacity.
- **Maintainability:** The degree to which the software can be modified, corrected, enhanced, or adapted to changing needs and environments. Maintainability can be measured by criteria such as modularity, readability, testability, analyzability, changeability, and stability.
- **Portability:** The degree to which the software can be transferred, installed, and executed on different platforms, systems, or environments. Portability can be measured by criteria such as adaptability, installability, coexistence, and replaceability.

Software Crisis

- Software crisis is a term used to describe the difficulty of writing useful and efficient computer programs in the required time.

- Software crisis was due to the rapid increases in computer power and the complexity of the problems that could not be tackled with the existing software development methods.
- Some of the symptoms of software crisis are:
 - The cost of owning and maintaining software was as expensive as developing the software.
 - Projects were running over-time and over-budget.
 - Software was very inefficient and unreliable.
 - The quality of the software was low and did not meet user requirements.
 - Software was difficult to modify and reuse.
- Some of the causes of software crisis are:
 - Poor project management and planning.
 - Lack of skilled and experienced developers.
 - Under-specified or changing requirements.
 - Inadequate testing and quality assurance.
 - Lack of standardization and documentation.
- Some of the solutions to software crisis are:
 - Adopting structured and modular programming techniques.
 - Using software engineering principles and methodologies.
 - Applying software quality metrics and models.
 - Developing reusable and maintainable software components.
 - Using automated tools and environments for software development and testing.

Software Engineering Processes

Software engineering processes are the methods and techniques used to develop and maintain software products that meet the client's requirements and expectations. Software engineering processes aim to improve the quality, efficiency, and reliability of software development and delivery.

Some of the common software engineering processes are:

- **Waterfall:** a linear, sequential approach to software development, with distinct phases such as requirements gathering, design, implementation, testing, and maintenance. Each phase must be completed and verified before moving to the next phase. This process is simple, easy to follow, and suitable for well-defined and stable projects. However, it is rigid, inflexible, and does not accommodate changes or feedback easily.
- **Agile:** an iterative, incremental, and adaptive approach to software development, with short cycles of development and delivery called sprints. Each sprint involves planning, analysis, design, coding, testing, and review. This process is flexible, responsive, and collaborative, and allows for frequent changes and feedback from the client and the users. However, it is complex, challenging, and requires high levels of commu-

nication and coordination among the team members and the stakeholders.

- **Lean:** a value-driven, waste-reducing, and customer-focused approach to software development, with principles such as eliminate waste, amplify learning, deliver fast, empower the team, build integrity, and optimize the whole. This process is efficient, effective, and innovative, and aims to maximize customer value and minimize waste. However, it is demanding, risky, and requires a strong culture of continuous improvement and experimentation.
- **Traditional/Waterfall:** a structured, disciplined, and formal approach to software development, with phases such as feasibility study, requirements analysis, system design, detailed design, coding, testing, integration, installation, and maintenance. Each phase has its own deliverables, documentation, and quality standards. This process is comprehensive, rigorous, and consistent, and ensures that the software meets the functional and non-functional requirements. However, it is slow, costly, and does not cope well with changes or uncertainties.

These are some of the software engineering processes that can be used to develop software products. Depending on the environmental, organizational, and product constraints, different processes may be more or less suitable for different projects. Software engineers and other experts should analyze the project's characteristics and choose the most appropriate process for their needs.

Similarity and Differences from Conventional Engineering Processes

- Conventional engineering processes are the methods and techniques used to design, develop, test, and deploy products or systems that meet the requirements and specifications of customers or stakeholders.
- Software engineering processes are the methods and techniques used to design, develop, test, and deploy software products or systems that meet the requirements and specifications of customers or stakeholders.
- **Similarity:** Both conventional and software engineering processes follow a systematic and disciplined approach to ensure the quality, reliability, and usability of the products or systems. Both processes involve activities such as planning, analysis, design, implementation, testing, maintenance, and evolution. Both processes also use tools, standards, models, and metrics to support the activities and measure the outcomes.
- **Differences:** Software engineering processes differ from conventional engineering processes in several aspects, such as:
 - The nature of the product or system: Software products or systems are intangible, complex, and dynamic, whereas conventional products or systems are tangible, simple, and static. Software products or systems are also more prone to errors, changes, and uncertainties than conventional products or systems.

- The nature of the requirements and specifications: Software requirements and specifications are often incomplete, inconsistent, ambiguous, and evolving, whereas conventional requirements and specifications are usually clear, stable, and well-defined. Software requirements and specifications also depend on the context, environment, and user feedback, whereas conventional requirements and specifications are more independent of these factors.
- The nature of the design and development: Software design and development are more iterative, incremental, and adaptive, whereas conventional design and development are more linear, sequential, and prescriptive. Software design and development also involve more creativity, abstraction, and experimentation than conventional design and development.
- The nature of the testing and deployment: Software testing and deployment are more continuous, automated, and distributed, whereas conventional testing and deployment are more discrete, manual, and centralized. Software testing and deployment also require more verification, validation, and evaluation than conventional testing and deployment.

Software Quality Attributes

Software quality attributes are the characteristics of a software system that affect its functionality, performance, usability, reliability, security, maintainability, portability, and other aspects. Software quality attributes are also known as non-functional requirements, quality characteristics, or quality factors.

Some examples of software quality attributes are:

- **Functionality:** The degree to which the software meets the specified requirements and provides the desired functions.
- **Performance:** The degree to which the software responds to user requests and executes tasks within acceptable time and resource limits.
- **Usability:** The degree to which the software is easy to learn, use, and understand by the intended users.
- **Reliability:** The degree to which the software operates correctly and consistently under normal and abnormal conditions.
- **Security:** The degree to which the software protects the data and resources from unauthorized access, modification, or damage.
- **Maintainability:** The degree to which the software can be modified, corrected, or improved by the developers or maintainers.
- **Portability:** The degree to which the software can be transferred, installed, or executed on different platforms, environments, or devices.

Software quality attributes are important for designing, developing, testing, and evaluating software systems. They help to define the quality goals and criteria for the software, and to measure and improve the quality of the software. Soft-

ware quality attributes also influence the trade-offs and decisions that need to be made during the software development process, as different attributes may have conflicting or complementary effects on each other. For example, improving the performance of a software system may reduce its portability, or enhancing the security of a software system may affect its usability. Therefore, software quality attributes need to be carefully balanced and prioritized according to the needs and expectations of the stakeholders and the users of the software system.

Software Development Life Cycle (SDLC) Models

Software Development Life Cycle (SDLC) is a process that describes the stages involved in developing a software product from its inception to its delivery. SDLC models are frameworks that define the tasks, activities, and deliverables for each phase of the software development process.

There are different types of SDLC models, each with its own advantages and disadvantages. Some of the common SDLC models are:

- **Waterfall model:** This is the most traditional and sequential model, where the software development process is divided into linear and fixed phases, such as requirement analysis, design, implementation, testing, deployment, and maintenance. Each phase must be completed and verified before moving to the next phase. The main advantage of this model is its simplicity and clarity, but the main disadvantage is its rigidity and lack of flexibility to accommodate changes or feedback during the development process.
- **Agile model:** This is a modern and iterative model, where the software development process is divided into short and incremental cycles, called sprints, each lasting from two to four weeks. Each sprint involves planning, designing, coding, testing, and reviewing the software product or a part of it. The main advantage of this model is its adaptability and responsiveness to changing requirements and customer feedback, but the main disadvantage is its difficulty in managing complex and large-scale projects.
- **Spiral model:** This is a hybrid model that combines the features of the waterfall and agile models. The software development process is divided into four quadrants: planning, risk analysis, engineering, and evaluation. Each quadrant is repeated in a circular and iterative manner until the software product meets the customer's expectations and needs. The main advantage of this model is its ability to handle complex and risky projects, but the main disadvantage is its high cost and time consumption.
- **V-model:** This is an extension of the waterfall model, where the software development process is divided into two parallel streams: verification and validation. Verification involves checking the quality and correctness of the software product at each phase of the development process, such as require-

ment analysis, design, coding, and testing. Validation involves checking the functionality and usability of the software product at the end of the development process, such as system testing, user acceptance testing, and deployment. The main advantage of this model is its emphasis on quality assurance and testing, but the main disadvantage is its lack of flexibility and feedback during the development process.

- **Prototyping model:** This is an experimental model, where the software development process involves creating a prototype or a mock-up of the software product or a part of it, based on the initial requirements and specifications. The prototype is then evaluated and refined by the customer and the developer until it meets the customer's expectations and needs. The main advantage of this model is its ability to elicit and validate the requirements and design of the software product, but the main disadvantage is its potential to create unrealistic and incomplete prototypes.

Waterfall Model in SDLC

The waterfall model is a linear, sequential approach to the software development lifecycle (SDLC) that is popular in software engineering and product development. The waterfall model uses a logical progression of SDLC steps for a project, similar to the direction water flows over the edge of a cliff.

The waterfall model is the earliest SDLC approach that was used for software development. The waterfall model illustrates the software development process in a linear sequential flow. This means that any phase in the development process begins only if the previous phase is complete.

The waterfall model consists of the following phases:

- **Requirement analysis:** In this phase, the project requirements are gathered and documented. The scope, objectives, and constraints of the project are defined. The feasibility and risks of the project are also analyzed.
- **System design:** In this phase, the system architecture and specifications are designed based on the requirements. The hardware and software requirements are also identified. The design documents are reviewed and approved by the stakeholders.
- **Implementation:** In this phase, the system components are coded, tested, and integrated according to the design specifications. The code quality and functionality are verified and validated. The system is also documented for future reference and maintenance.
- **Testing:** In this phase, the system is tested as a whole to ensure that it meets the requirements and expectations of the users and stakeholders. The system is checked for errors, bugs, and defects. The system is also tested for performance, security, and usability.
- **Deployment:** In this phase, the system is deployed to the production environment and made available to the end-users. The system is also monitored and maintained for any issues or feedback. The system is also

updated and enhanced as per the changing needs of the users and stakeholders.

- **Maintenance:** In this phase, the system is supported and maintained for its entire lifecycle. The system is also modified and improved as per the changing requirements and feedback. The system is also retired or replaced when it becomes obsolete or inefficient.

Some of the advantages and disadvantages of the waterfall model are:

- Advantages:
 - The waterfall model is simple and easy to understand and use.
 - The waterfall model provides a clear and structured way of developing software.
 - The waterfall model ensures that the requirements are well-defined and stable before the development begins.
 - The waterfall model facilitates quality control and management as each phase has specific deliverables and review processes.
 - The waterfall model is suitable for projects that have fixed and clear requirements and specifications, and that do not require frequent changes or iterations.
- Disadvantages:
 - The waterfall model is rigid and inflexible, and does not accommodate changes or revisions easily.
 - The waterfall model does not allow feedback or testing until the later stages of the development, which can lead to errors and defects that are costly and time-consuming to fix.
 - The waterfall model does not involve the users or stakeholders much in the development process, which can result in a system that does not meet their needs or expectations.
 - The waterfall model can be risky and inefficient, as it assumes that the requirements and design are correct and complete, and that there are no uncertainties or complexities in the project.
 - The waterfall model is not suitable for projects that have dynamic and evolving requirements and specifications, and that require frequent changes or iterations.

Prototype Model in SDLC

- The prototype model is a software development life cycle (SDLC) model in which a prototype is built, tested, and then reworked as necessary until an acceptable prototype is finally achieved from which the complete system or product can be developed.
- A prototype is a working model of a software or system that displays the functionality of the product under development, but may not contain the exact logic or implementation details of the original software.
- The prototype model is used when the customers do not know the exact project requirements beforehand or when the requirements are complex

and unclear.

- The prototype model involves the following phases :
 - **Requirement gathering and analysis:** The basic requirements of the project are gathered from the customer and analyzed. The scope and feasibility of the project are also determined in this phase.
 - **Quick design:** A quick design of the prototype is created based on the requirements. The design does not have to be detailed or complete, but it should capture the essential features and functionality of the product.
 - **Build prototype:** The prototype is built according to the quick design. The prototype can be a paper-based model, a mock-up, a simulation, or a partial implementation of the software. The prototype should be easy to modify and refine.
 - **Customer evaluation:** The prototype is presented to the customer for feedback and evaluation. The customer can test the prototype and suggest changes or improvements. The customer can also validate if the prototype meets their expectations and requirements.
 - **Refining prototype:** Based on the customer feedback, the prototype is refined and improved. The changes are incorporated into the prototype and the cycle of building, testing, and evaluation is repeated until the customer is satisfied with the prototype.
 - **Engineer product:** Once the customer approves the prototype, the final product is engineered based on the prototype. The prototype serves as a basis for the design, development, testing, and deployment of the product.
- The advantages of the prototype model are :
 - It reduces the risk of failure and misunderstanding of the requirements by involving the customer in the development process.
 - It improves the quality and usability of the product by incorporating the customer feedback and suggestions.
 - It facilitates better communication and collaboration between the developers and the customer.
 - It allows for early detection and correction of errors and defects.
 - It increases the customer satisfaction and confidence in the product.
- The disadvantages of the prototype model are :
 - It can be time-consuming and costly to build and refine multiple prototypes.
 - It can lead to scope creep and unrealistic expectations from the customer if the requirements are not well-defined and controlled.
 - It can cause confusion and inconsistency between the prototype and the final product if the prototype is not properly documented and managed.
 - It can compromise the quality and performance of the product if the prototype is not properly tested and verified.

Spiral Model in SDLC

- Spiral model is a software development life cycle (SDLC) model that provides a systematic and iterative approach to software development .
- Spiral model is a risk-driven process model, which means the overall project's success depends on the risk analysis phase .
- Spiral model is one of the oldest types of SDLC, and is favored for large, expensive, and complicated projects .
- Spiral model consists of four phases: planning, risk analysis, engineering, and evaluation .
- Spiral model is based on the idea of a spiral, with each iteration of the spiral representing a complete software development cycle, from requirements gathering and analysis to design, implementation, testing, and deployment .
- Spiral model supports changing requirements and feedback, as each iteration of the spiral can be modified based on the customer's needs and expectations .
- Spiral model also supports prototyping, as each iteration of the spiral can produce a working version of the software that can be demonstrated to the customer and evaluated for feedback .
- Spiral model has some advantages, such as:
 - It can handle complex and large projects effectively .
 - It can reduce risks and uncertainties by conducting risk analysis at each iteration .
 - It can accommodate changing requirements and feedback easily .
 - It can produce high-quality software by ensuring testing and evaluation at each iteration .
- Spiral model also has some disadvantages, such as:
 - It can be costly and time-consuming due to the extensive risk analysis and documentation involved .
 - It can be difficult to manage and control due to the lack of a clear end point and the possibility of scope creep .
 - It can require highly skilled and experienced developers and managers who can handle the risks and complexities involved .

Evolutionary Development Models in SDLC

- Evolutionary Development Models are a type of Software Development Life Cycle (SDLC) models that aim to deliver software products in successive versions, each with more features and functionality than the previous one.
- Evolutionary Development Models are based on the principle of feedback and adaptation, where the software is continuously refined and improved based on the customer's needs and expectations.
- Evolutionary Development Models are suitable for projects that have un-

clear or changing requirements, where the customer is involved throughout the development process, and where the software needs to be delivered quickly and incrementally.

- Evolutionary Development Models can be classified into two types: **Iterative** and **Incremental**.
 - **Iterative Development Model** is a type of Evolutionary Development Model where the software is developed in cycles, each cycle consisting of planning, analysis, design, implementation, testing, and evaluation. Each cycle produces a working version of the software, which is then reviewed and modified based on the feedback from the customer and the developers. The cycle is repeated until the software meets the customer's satisfaction and quality standards.
 - **Incremental Development Model** is a type of Evolutionary Development Model where the software is developed in modules, each module adding some functionality to the software. Each module is developed using a mini-waterfall model, where the requirements, design, implementation, testing, and integration are done for each module separately. The modules are then integrated into the software, which is then delivered to the customer. The process is repeated until all the modules are completed and the software is fully functional.
- Evolutionary Development Models have some advantages and disadvantages over other SDLC models.
 - Advantages of Evolutionary Development Models include:
 - * They allow for flexibility and adaptability to changing requirements and customer feedback.
 - * They reduce the risk of delivering a software product that does not meet the customer's needs or expectations.
 - * They enable faster delivery of software products and early detection of errors and defects.
 - * They facilitate customer involvement and satisfaction throughout the development process.
 - * They support object-oriented software development, where the software can be easily partitioned into units or objects.
 - Disadvantages of Evolutionary Development Models include:
 - * They require more management and coordination among the developers and the customer.
 - * They may result in scope creep, where the software becomes more complex and costly than initially planned.
 - * They may compromise the quality and consistency of the software, as the software may not undergo rigorous testing and documentation.

- * They may not be suitable for large-scale or safety-critical software projects, where the requirements and specifications need to be clearly defined and verified.

Iterative Enhancement Models in SDLC

- Iterative Enhancement Models are a type of software development life cycle (SDLC) models that involve building software incrementally by adding more functionality in each iteration .
- In this model, the software is divided into smaller modules or components, each of which can be developed and tested separately .
- Each component is treated as a sub-project and goes through all the phases of the SDLC, such as planning, analysis, design, implementation, testing, and deployment .
- The first component is usually a basic or core version of the software that satisfies some of the essential requirements .
- The subsequent components add more features and enhancements to the software, based on the feedback from the users and stakeholders .
- The process of developing and integrating components is repeated until the software meets all the desired requirements and is ready for delivery .
- Iterative Enhancement Models are also known as Incremental Models or Iterative and Incremental Development (IID) Models .

Unit 2 - Software Requirement Specifications (SRS)

- Software Requirement Specifications (SRS) is a document that describes the features, functions, and constraints of a software system.
- SRS is also known as Software Requirements Document (SRD), Software Requirements Specification (SRS), or Functional Specification (FS).
- SRS is a communication tool between the stakeholders and the developers of the software system.
- SRS helps to ensure that the software system meets the needs and expectations of the stakeholders and the users.
- SRS also serves as a basis for software design, testing, validation, verification, maintenance, and evolution.
- The main components of SRS are:
 - Introduction: This section provides an overview of the software system, its purpose, scope, objectives, and assumptions.
 - User Requirements: This section describes the needs and expectations of the intended users of the software system, such as their goals, tasks, preferences, and constraints.
 - System Requirements: This section specifies the functional and non-functional requirements of the software system, such as its features,

functions, performance, reliability, security, usability, and compatibility.

- **System Models:** This section presents the conceptual, logical, and physical models of the software system, such as use cases, data flow diagrams, entity-relationship diagrams, class diagrams, and state diagrams.
- **System Specification:** This section defines the detailed design and implementation specifications of the software system, such as its architecture, components, interfaces, algorithms, data structures, and protocols.
- **Appendices:** This section contains any additional information that supports the SRS, such as glossary, references, acronyms, and abbreviations.

Requirement Engineering Process in SRS

Requirement engineering is the process of eliciting, analyzing, specifying, and validating the software requirements for a system. It is an essential and crucial activity in software development, as it lays the foundation for the design, implementation, testing, and maintenance of the software product. The main goal of requirement engineering is to produce a high-quality software requirements specification (SRS) document that describes the functional and non-functional requirements of the system, as well as the constraints, assumptions, and dependencies that affect the system.

The requirement engineering process consists of the following main activities:

- **Feasibility study:** This is the initial stage of the requirement engineering process, where the feasibility of the proposed system is assessed in terms of technical, economic, legal, operational, and schedule aspects. The feasibility study helps to determine whether the system is worth developing, and what are the possible risks and benefits involved. The feasibility study also defines the scope and objectives of the system, and identifies the main stakeholders and their needs.
- **Requirement elicitation:** This is the process of gathering the software requirements from various sources, such as the stakeholders, existing systems, documents, standards, regulations, etc. Requirement elicitation involves various techniques, such as interviews, questionnaires, surveys, brainstorming, observation, prototyping, etc. The aim of requirement elicitation is to collect as much information as possible about the system and its context, and to understand the problems and expectations of the users and customers.
- **Requirement analysis:** This is the process of analyzing the collected requirements and resolving any conflicts, ambiguities, inconsistencies, or gaps among them. Requirement analysis also involves prioritizing the requirements based on their importance, urgency, and dependency. Requirement analysis helps to refine and structure the requirements, and to

identify the functional and non-functional requirements of the system. Requirement analysis also defines the acceptance criteria and the verification and validation methods for the requirements.

- **Requirement specification:** This is the process of documenting the software requirements in a formal and precise manner, using a standard notation or language. Requirement specification produces the software requirements specification (SRS) document, which is the main output of the requirement engineering process. The SRS document serves as a contract between the developers and the customers, and as a basis for the design, implementation, testing, and maintenance of the system. The SRS document should be clear, complete, consistent, verifiable, modifiable, and traceable.
- **Requirement validation:** This is the process of checking the quality and correctness of the software requirements, and ensuring that they meet the needs and expectations of the stakeholders. Requirement validation involves reviewing, inspecting, testing, and evaluating the software requirements, and identifying and resolving any errors, defects, or discrepancies in the SRS document. Requirement validation also involves obtaining feedback and approval from the stakeholders, and ensuring that the requirements are aligned with the project scope and objectives.

Elicitation in Requirement Engineering Process in SRS

- Elicitation is the process of gathering, analyzing, and validating the needs and expectations of the stakeholders for a software system.
- Elicitation is one of the main activities in the requirement engineering process, which aims to produce a complete, consistent, and unambiguous specification of the software requirements.
- Elicitation involves various techniques to collect information from different sources, such as interviews, questionnaires, surveys, observation, prototyping, brainstorming, etc.
- Elicitation also involves resolving conflicts, clarifying ambiguities, and prioritizing requirements among the stakeholders.
- Elicitation is an iterative and collaborative process that requires effective communication and negotiation skills from the requirement engineers and the stakeholders.
- Elicitation is a challenging and critical task, as it affects the quality and success of the software system. Poor elicitation can lead to incomplete, inconsistent, or irrelevant requirements, which can cause delays, errors, or failures in the software development.

Analysis in Requirement Engineering Process in SRS

- Analysis is the second phase of the requirement engineering process, after elicitation.
- Analysis involves examining, evaluating, and refining the elicited require-

ments to ensure that they are consistent, complete, correct, and feasible.

- Analysis also involves resolving any conflicts or ambiguities among the stakeholders' needs and expectations, and prioritizing the requirements based on their importance and urgency.
- Analysis produces a set of validated and prioritized requirements that can be used as the basis for the design and development of the software system.
- Analysis can be performed using various techniques, such as:
 - Modeling: using graphical or textual representations to describe the structure, behavior, and interactions of the system and its components.
 - Prototyping: creating a simplified or partial version of the system to demonstrate its functionality and usability to the stakeholders and obtain their feedback.
 - Verification: checking that the requirements conform to the standards and specifications, and that they are free of errors and inconsistencies.
 - Validation: confirming that the requirements meet the stakeholders' needs and expectations, and that they are realistic and achievable.
 - Prioritization: assigning a level of importance and urgency to each requirement, based on the criteria and constraints of the project.
 - Negotiation: resolving any conflicts or trade-offs among the requirements and the stakeholders, and reaching a mutual agreement on the final set of requirements.

Documentation in Requirement Engineering Process in SRS

- Documentation is the process of recording the requirements and specifications of a software system in a written form.
- Documentation is an essential part of the requirement engineering process, as it helps to communicate, verify, validate, and manage the requirements of the software system.
- Documentation also serves as a contract between the stakeholders and the developers, as it defines the scope, functionality, quality, and constraints of the software system.
- Documentation can be done at different levels of abstraction, such as user, system, and component level, depending on the intended audience and purpose of the document.
- Documentation can follow different formats, standards, and templates, such as IEEE 830, IEEE 29148, or Volere, to ensure consistency, completeness, and clarity of the requirements.
- Documentation can include different types of information, such as:
 - Functional requirements: describe what the software system should do, such as features, use cases, scenarios, and user stories.

- Non-functional requirements: describe how the software system should behave, such as performance, reliability, security, usability, and maintainability.
- Constraints: describe the limitations and assumptions of the software system, such as budget, schedule, resources, regulations, and standards.
- Stakeholder needs: describe the expectations and goals of the stakeholders, such as customers, users, managers, and developers.
- Glossary: define the terms and acronyms used in the document.
- Models: illustrate the requirements using graphical or textual representations, such as diagrams, tables, charts, and equations.
- Traceability: link the requirements to their sources, dependencies, and verification methods, such as interviews, surveys, prototypes, and tests.

Review and Management of User Needs in Requirement Engineering Process in SRS

- Requirement engineering (RE) is the process of eliciting, analyzing, specifying, validating, and managing the requirements of a software system.
- User needs are the expectations, preferences, goals, and problems of the intended users of the software system.
- Review and management of user needs are important activities in RE to ensure that the software system meets the user needs and satisfies the user requirements.
- Review of user needs involves checking the completeness, consistency, correctness, clarity, and feasibility of the user needs, as well as resolving any conflicts or ambiguities among them.
- Management of user needs involves prioritizing, tracing, and controlling the changes of the user needs throughout the RE process and the software development life cycle.
- Some of the techniques and tools for review and management of user needs are:
 - Interviews, surveys, questionnaires, observations, and focus groups for eliciting and validating user needs.
 - Use cases, scenarios, user stories, and personas for specifying and analyzing user needs.
 - Prototypes, mockups, and wireframes for demonstrating and verifying user needs.
 - Requirement specification documents, such as software requirement specification (SRS), for documenting and communicating user needs.
 - Requirement traceability matrix (RTM), for linking user needs to user requirements, design specifications, test cases, and code modules.
 - Requirement management software, such as Jira, Trello, or Rational DOORS, for organizing, tracking, and updating user needs and their status.

Feasibility Study in Software Requirement Specification (SRS)

- A feasibility study is a study conducted to ascertain the viability of a structure or strategy.
- It is also one of the requirements of the engineering process.
- It is required when a client proposes a company to develop a product with a rough understanding of software functions and what functionality they necessitate from the software.
- The feasibility study aims to ascertain why developing software is appealing to users, adaptable to change, and compliant with applicable requirements .
- It delves into technical aspects of the project and product, such as compatibility, maintainability, efficiency, and integration capabilities.
- The feasibility study, especially professional software project feasibility analysis, takes place after specifying business requirements, that is, it is the second step of the requirements engineering process.
- Its justification is justified in order to analyze and answer some questions from the point of view of operational, technical, schedule, economic, and legal feasibility.
- The feasibility study helps to identify the risks involved in the project and to determine whether the project is worth pursuing or not.
- The feasibility study also helps to define the scope and objectives of the project, as well as the constraints and assumptions.
- The feasibility study can be documented in a Software Requirement Specification (SRS) document, which is a formal statement of the software requirements for the project.
- The SRS document serves as a basis for communication between the stakeholders, the developers, and the testers of the software.
- The SRS document should be clear, concise, consistent, complete, correct, and verifiable.
- The SRS document should also include the following sections:
 - Introduction: This section provides an overview of the purpose, scope, definitions, acronyms, abbreviations, references, and overview of the document.
 - General Description: This section provides a general description of the software, including the product perspective, product functions, user characteristics, general constraints, assumptions and dependencies, and requirements apportionment.
 - Specific Requirements: This section provides a detailed description of the functional and non-functional requirements of the software, such as performance, reliability, security, usability, maintainability, portability, etc. This section also includes the external interface requirements, such as user, hardware, software, and communication interfaces.
 - Appendices: This section provides any additional information that is relevant to the software requirements, such as data models, data

dictionaries, glossaries, etc.

Information Modelling in Software Requirement Specification (SRS)

- Information modelling is the process of creating an abstract, formal representation of the data and concepts that are relevant to a specific domain or problem.
- Information modelling helps to define the structure, semantics, constraints, and operations of the data and concepts in the domain.
- Information modelling is an essential part of software requirement specification (SRS), which is a document that describes what the software will do and how it will perform.
- SRS is an official document that shows the details of the expected system, such as its functionality, performance, quality, and interface requirements.
- SRS is created before starting the development work, and it serves as a blueprint or roadmap for the software development team and the customer.
- Information modelling in SRS can be done using various techniques, such as entity-relationship diagrams, class diagrams, data flow diagrams, state transition diagrams, etc.
- Information modelling in SRS can help to achieve the following benefits:
 - It can improve the communication and understanding between the stakeholders, such as the developers, customers, users, and testers.
 - It can reduce the ambiguity and inconsistency of the requirements, and make them more precise and verifiable.
 - It can facilitate the analysis, design, implementation, testing, and maintenance of the software system.
 - It can support the reuse and integration of existing data and concepts from other sources or systems.

Data Flow Diagrams in Software Requirement Specification (SRS)

- Data flow diagrams (DFD) are graphical representations of the flow of data and information in a system or process.
- DFDs are used to model the functional requirements of a software system, showing how data is input, processed, stored, and output by different components or modules.
- DFDs are also used to communicate the requirements to the stakeholders, customers, and developers, as they are easy to understand and use simple symbols and syntax.
- DFDs can be drawn at different levels of abstraction, depending on the level of detail and complexity required. The most common levels are:

- 0-level DFD: Also known as the context diagram, it shows the entire system as a single process, with its inputs and outputs to the external environment.
- 1-level DFD: It shows the main processes or functions of the system, and how they are connected by data flows. It also shows the data sources and destinations, such as files, databases, or external entities.
- 2-level DFD: It shows the sub-processes or modules of each process or function, and how they interact with each other and with the data. It also shows the data stores and data structures used by the system.
- DFDs are useful for:
 - Identifying the scope and boundaries of the system, and the data and information needs of the users and stakeholders.
 - Analyzing and decomposing the system into smaller and manageable units, and identifying the dependencies and relationships among them.
 - Verifying and validating the requirements, and detecting any errors, inconsistencies, or ambiguities.
 - Documenting the requirements in a structured and standardized format, and facilitating the design and implementation of the system.

Entity Relationship Diagrams in Software Requirement Specification (SRS)

- Entity Relationship Diagrams (ERDs) are graphical representations of the data model of a software system.
- ERDs show the entities, attributes, relationships and constraints of the data that the software system needs to store and manipulate.
- ERDs are used in software engineering during the planning stages of the software project to analyze the requirements and design the database of the software system.
- ERDs are often used as the basis for data flow diagrams (DFDs) that show how the data flows between the components of the software system.
- ERDs are composed of the following elements:
 - Entities: An entity is a person, place, thing or concept that the software system needs to keep information about. For example, a student, a course, a book, etc. Entities are represented by rectangles in ERDs.
 - Attributes: An attribute is a property or characteristic of an entity that the software system needs to store. For example, a student entity may have attributes such as name, ID, email, etc. Attributes are represented by ovals connected to the entity they belong to in ERDs.
 - Relationships: A relationship is an association or link between two or more entities that the software system needs to capture. For example, a student entity may have a relationship with a course entity that

indicates the student is enrolled in the course. Relationships are represented by diamonds or lines connecting the entities they involve in ERDs.

- Constraints: A constraint is a rule or restriction that applies to the entities, attributes or relationships in the data model. For example, a constraint may specify that a student can only enroll in one course at a time, or that a book can only have one author. Constraints are represented by symbols or text annotations in ERDs.

Decision Tables in Software Requirement Specification (SRS)

- A decision table is a tabular representation of the logic and rules that govern the behavior of a software system.
- A decision table consists of four quadrants: condition stubs, condition entries, action stubs, and action entries.
- Condition stubs are the variables or factors that affect the outcome of a decision. Condition entries are the possible values or states of the condition stubs.
- Action stubs are the actions or outcomes that result from a decision. Action entries are the indicators that show which actions are performed for a given combination of condition entries.
- A decision table can have one or more rules, which are the horizontal rows that link the condition entries and the action entries.
- A decision table can be used to specify the functional requirements of a software system in a clear, concise, and consistent way.
- A decision table can also be used to verify the completeness and correctness of the requirements, by checking for missing, conflicting, or redundant rules.
- A decision table can be converted into other forms of representation, such as pseudocode, flowcharts, or decision trees, for implementation or documentation purposes.

SRS Document

- A software requirements specification (SRS) document is a document that describes what the software will do and how it will be expected to perform .
- It also describes the functionality the product needs to fulfill the needs of all stakeholders (business, users) .
- An SRS document can be thought of as a blueprint or roadmap for the software to be built.
- A typical SRS document lays out all software requirements in detail and sometimes even includes a set of use cases that describe the software's necessary user interactions.
- It describes a software project's purpose, contains an overall description of its features, and specifies its requirements.

- An SRS document can help to ensure that the software meets the expectations of the stakeholders, reduces the development costs and risks, and facilitates communication and collaboration among the project team .
- An SRS document should be clear, concise, consistent, complete, correct, and verifiable .
- An SRS document should follow a standard structure and format, such as the IEEE 830-1998 standard .
- An SRS document should be updated and maintained throughout the software development life cycle .

IEEE Standards for SRS

- A software requirements specification (SRS) is a description of a software system to be developed. It is modeled after business requirements specification (CONOPS) .
- The IEEE standard 830, last revised in 1998, has since been replaced by standard ISO/IEC/IEEE 29148:2011, with an update in 2018 . This standard covers the processes and information it recommends for an SRS document, as well as its format .
- The content and qualities of a good SRS are described in the standard. A good SRS should be:
 - Correct: It should state the true requirements of the system.
 - Unambiguous: It should have only one possible interpretation.
 - Complete: It should cover all the requirements of the system and not omit any essential information.
 - Consistent: It should not contain any conflicting or contradictory requirements.
 - Ranked: It should prioritize the requirements based on their importance, urgency, or stability.
 - Verifiable: It should be possible to check whether the system meets the requirements.
 - Modifiable: It should be easy to update and maintain as the system evolves.
 - Traceable: It should be possible to trace the origin and impact of each requirement.
- The format and organization of an SRS document can vary depending on the project and the preferences of the stakeholders. However, the standard suggests some common elements that should be included in an SRS document, such as:
 - Introduction: It should provide an overview of the purpose, scope, definitions, acronyms, abbreviations, references, and overview of the document.
 - Overall description: It should describe the general factors that affect the system and its requirements, such as product perspective, product functions, user characteristics, constraints, assumptions, and dependencies.

- Specific requirements: It should state all the functional and nonfunctional requirements of the system in detail, using one or more of the following criteria: mode, user class, object, feature, stimulus, functional hierarchy, or combinations of these criteria .
- Appendices: It should provide any additional information that is relevant to the SRS document, such as index, glossary, bibliography, or change history.
- Index: It should list all the terms, symbols, abbreviations, and acronyms used in the document, along with their definitions and references.

Software Quality Assurance (SQA) in SRS

- Software Quality Assurance (SQA) is the process of ensuring that the software products and processes meet the specified quality standards and requirements.
- SQA involves planning, implementing, monitoring, and improving the quality activities throughout the software development life cycle (SDLC).
- SQA aims to prevent defects, reduce rework, improve customer satisfaction, and enhance software quality and reliability.
- SQA in SRS refers to the quality assurance activities that are performed during the software requirements specification (SRS) phase of the SDLC.
- SRS is the document that defines the functional and non-functional requirements of the software system to be developed.
- SQA in SRS includes the following tasks:
 - Reviewing the SRS document for completeness, consistency, clarity, correctness, and testability.
 - Verifying that the SRS document conforms to the applicable standards, guidelines, and best practices.
 - Validating that the SRS document reflects the needs and expectations of the stakeholders and users.
 - Identifying and resolving any issues, ambiguities, or conflicts in the SRS document.
 - Tracking and managing the changes and revisions to the SRS document.
 - Conducting audits and inspections to ensure the quality of the SRS document and process.
 - Reporting and documenting the results and outcomes of the SQA in SRS activities.

Verification and Validation in SRS

Verification and validation are two important activities in the software requirements engineering process. They aim to ensure that the software requirements specification (SRS) document is complete, consistent, correct, realistic, unambiguous and verifiable.

- Verification is the process of checking whether the SRS document conforms to the standards, guidelines and specifications established for it. It answers the question: “Are we building the product right?” Verification can be done by using various methods, such as peer reviews, walkthroughs, inspections, etc. Verification helps to detect errors, omissions, inconsistencies and ambiguities in the SRS document at an early stage of development.
- Validation is the process of checking whether the SRS document meets the actual needs and expectations of the stakeholders. It answers the question: “Are we building the right product?” Validation can be done by using various methods, such as prototyping, simulation, testing, etc. Validation helps to ensure that the software system will perform its intended functions and satisfy the user requirements.

Verification and validation should be done continuously throughout the development of requirements at every level and as part of baseline activities and reviewed during the System Requirements Reviews (SRR). Verification and validation help to improve the quality of the software system and reduce the risks of failure and rework.

SQA Plans in SRS

- SQA stands for Software Quality Assurance, which is the process of ensuring that the software meets the specified requirements and standards of quality.
- SRS stands for Software Requirements Specification, which is the document that describes the functional and non-functional requirements of the software system.
- SQA plans are the documents that outline the activities, roles, responsibilities, tools, techniques, and standards for ensuring the quality of the software throughout the development life cycle.
- SQA plans are usually derived from the SRS, as the SRS defines the quality attributes and criteria that the software must satisfy.
- SQA plans typically include the following sections:
 - Introduction: This section provides the background, scope, purpose, and objectives of the SQA plan.

- SQA tasks: This section describes the tasks that will be performed by the SQA team or personnel, such as reviews, audits, inspections, testing, verification, validation, etc.
- SQA roles and responsibilities: This section defines the roles and responsibilities of the SQA team or personnel, such as SQA manager, SQA engineer, SQA auditor, etc.
- SQA schedule: This section specifies the time frame and milestones for the SQA activities, such as when and how often the reviews, audits, inspections, testing, etc. will be conducted.
- SQA resources: This section identifies the resources that will be used for the SQA activities, such as tools, techniques, methods, standards, guidelines, checklists, etc.
- SQA standards: This section defines the standards that will be followed for the SQA activities, such as IEEE, ISO, CMMI, etc.
- SQA documentation: This section describes the documentation that will be produced and maintained for the SQA activities, such as SQA plan, SQA report, SQA checklist, SQA metrics, etc.
- SQA risks and issues: This section identifies the potential risks and issues that may affect the quality of the software, such as changes in requirements, defects, delays, etc. and how they will be managed and resolved.
- SQA evaluation and improvement: This section describes the methods and criteria for evaluating and improving the effectiveness and efficiency of the SQA activities, such as feedback, measurement, analysis, etc.

Software Quality Frameworks (SQF) in SRS

- Software Quality Frameworks (SQF) are the set of principles, guidelines, and best practices that aim to ensure the quality of software products and services.
- SQF can be applied at different stages of the software development life cycle, such as planning, analysis, design, implementation, testing, deployment, and maintenance.
- SQF can help to define the quality requirements, criteria, and metrics for the software system, as well as the methods and tools for measuring, monitoring, and improving the quality.
- SQF can also help to establish the roles and responsibilities of the stakeholders involved in the software quality assurance (SQA) process, such as developers, testers, customers, and managers.
- SQF can be based on various standards, models, and frameworks that have been developed and adopted by the software industry, such as ISO 5055, IEEE 830, CMMI, and Agile.
- SQF can be tailored to the specific needs and characteristics of the software project, such as the size, complexity, domain, and methodology of the software system.

- SQF can be documented in the software requirements specification (SRS) document, which is a formal and comprehensive description of the software system to be developed .
- SQF can help to ensure that the SRS document is complete, consistent, clear, verifiable, and traceable, which are some of the quality characteristics of a good SRS.
- SQF can also help to align the expectations and objectives of the software project among the stakeholders, and to facilitate the communication and collaboration among the teams.
- SQF can ultimately contribute to the delivery of high-quality software products and services that meet the customer's needs and expectations, and that are reliable, secure, efficient, and maintainable.

ISO 9000 Models in SRS

- ISO 9000 is a family of standards that provide guidelines and principles for quality management systems (QMS) .
- QMS are the organizational processes and procedures that ensure the quality of products and services .
- ISO 9000 standards are applicable to any organization, regardless of size, type, or industry .
- ISO 9000 standards are based on the following seven quality management principles :
 - Customer focus: understanding and meeting customer needs and expectations
 - Leadership: establishing a vision, direction, and culture for the organization
 - Engagement of people: empowering and involving employees and stakeholders
 - Process approach: managing activities as interrelated processes
 - Improvement: pursuing continual enhancement of performance and outcomes
 - Evidence-based decision making: using data and information to support decisions
 - Relationship management: building and maintaining beneficial relationships with suppliers and partners
- ISO 9001 is the most widely used standard in the ISO 9000 family. It specifies the requirements for a QMS that an organization must fulfill to demonstrate its ability to consistently provide products and services that meet customer and regulatory requirements .
- ISO 9000-3 is a specific standard that provides guidance for the application of ISO 9001 to the development, supply, installation, and maintenance of computer software .
- Software Requirements Specification (SRS) is a document that describes the features, functions, and constraints of a software system. It is a key input for the software development process .

- SRS should follow the ISO/IEC/IEEE 29148 standard, which defines the content and structure of a good SRS .
- SRS should include the following sections :
 - Introduction: provides the purpose, scope, definitions, acronyms, abbreviations, references, and overview of the SRS
 - Overall description: provides the context, perspective, functions, user characteristics, constraints, assumptions, and dependencies of the software system
 - Specific requirements: provides the detailed functional, non-functional, performance, security, usability, reliability, maintainability, and portability requirements of the software system
 - Appendices: provides any additional information that is relevant to the SRS, such as use cases, data models, user interface mockups, etc.
 - Index: provides an alphabetical list of terms and topics covered in the SRS

SEI-CMM Model in SRS

- SEI-CMM stands for Software Engineering Institute - Capability Maturity Model.
- It is a framework that assesses and improves the software development processes of an organization.
- It defines five levels of process maturity, from initial to optimized, based on the presence and effectiveness of key process areas (KPAs).
- The KPAs are a set of goals, practices, and measures that describe the essential elements of a software process.
- The SRS (Software Requirements Specification) is a document that specifies the functional and non-functional requirements of a software system.
- The SRS should be developed following the best practices of the SEI-CMM model, such as:
 - Establishing a common understanding of the requirements among the stakeholders.
 - Managing the changes and traceability of the requirements.
 - Verifying and validating the requirements for completeness, consistency, and feasibility.
 - Applying quality standards and metrics to the requirements.
 - Using appropriate tools and techniques to elicit, analyze, specify, and document the requirements.
- The benefits of applying the SEI-CMM model to the SRS are:
 - Improved customer satisfaction and trust.
 - Reduced rework and defects.
 - Increased productivity and efficiency.
 - Enhanced project management and control.
 - Higher quality and reliability of the software system.

Unit 3 - Software Design

- Software design is the process of defining the architecture, components, interfaces, and other characteristics of a software system.
- Software design is based on the requirements analysis and the software development methodology chosen for the project.
- Software design aims to achieve the following goals:
 - Functional correctness: the software system should meet the functional and non-functional requirements specified by the stakeholders.
 - Reliability: the software system should perform consistently and correctly under normal and abnormal conditions.
 - Efficiency: the software system should make optimal use of the available resources, such as memory, CPU, disk, network, etc.
 - Maintainability: the software system should be easy to modify, debug, test, and evolve over time.
 - Usability: the software system should be easy to use, learn, and understand by the intended users.
 - Portability: the software system should be able to run on different platforms, environments, and devices with minimal changes.
 - Reusability: the software system should be able to reuse existing components, libraries, frameworks, or patterns to reduce development time and cost.
 - Scalability: the software system should be able to handle increasing workload, users, or data without degrading performance or quality.
 - Security: the software system should protect the confidentiality, integrity, and availability of the data and resources from unauthorized access, modification, or damage.
- Software design can be performed at different levels of abstraction, such as:
 - Architectural design: the high-level structure and organization of the software system, including the main components, their interactions, and their responsibilities.
 - Component design: the detailed specification and implementation of the individual components, such as classes, modules, functions, etc.
 - Interface design: the definition and documentation of the inputs, outputs, and behavior of the components, such as parameters, return values, exceptions, etc.
 - Algorithm design: the selection and optimization of the algorithms, data structures, and techniques used to solve the problems or tasks within the components.
 - User interface design: the design of the graphical, textual, or auditory elements that the users interact with, such as menus, buttons, dialogs, etc.
- Software design can be influenced by various factors, such as:
 - Software quality attributes: the desired properties or characteristics of the software system, such as performance, reliability, usability, etc.

- Software design principles: the general guidelines or best practices for creating good software design, such as modularity, cohesion, coupling, abstraction, encapsulation, etc.
- Software design patterns: the reusable solutions or templates for common or recurring problems or situations in software design, such as singleton, observer, factory, etc.
- Software design models: the graphical or textual representations or notations for describing the software design, such as UML, ERD, DFD, etc.
- Software design tools: the software applications or environments that support the software design process, such as IDEs, editors, debuggers, etc.

Basic Concept of Software Design

- Software design is the process of envisioning and defining software solutions to one or more sets of problems .
- Software design is based on the software requirements analysis (SRA), which is a part of the software development process that lists specifications used in software engineering.
- Software design is the process by which an agent creates a specification of a software artifact intended to accomplish goals, using a set of primitive components and subject to constraints.
- Software design involves the following activities :
 - Identifying the software requirements and constraints
 - Developing a software architecture that defines the components, interfaces, and interactions
 - Choosing or creating appropriate algorithms, data structures, and design patterns
 - Documenting and communicating the software design using diagrams, models, and notations
 - Evaluating and testing the software design for quality attributes such as functionality, reliability, usability, efficiency, maintainability, and portability
- Software design can be classified into three basic types:
 - Application software design, which focuses on creating software that provides specific functions or services to users or other software
 - System software design, which focuses on creating software that manages the hardware, operating system, and network resources
 - Embedded software design, which focuses on creating software that controls the behavior and functionality of devices or systems that are not typically considered computers

Architectural Design in Software Design

- Architectural design in software design is the process of defining a high-level structure and organization of a software system, as well as the components, interfaces, and interactions that constitute it .
- Architectural design is important because it provides a blueprint for the software development, guides the allocation of resources and responsibilities, facilitates communication and collaboration among stakeholders, and enables quality attributes such as performance, reliability, security, and maintainability .
- Architectural design follows a series of steps, such as:
 - Understanding the requirements and constraints of the software system
 - Identifying the key architectural drivers and quality attributes
 - Choosing an appropriate architectural style or pattern that suits the system's goals and context
 - Decomposing the system into components and defining their interfaces and interactions
 - Documenting and evaluating the architectural design using diagrams, models, and metrics .
- Architectural design can be influenced by various factors, such as:
 - The type and complexity of the software system
 - The domain and application of the software system
 - The available technologies and platforms
 - The standards and regulations that apply to the software system
 - The preferences and experiences of the software architects and developers .
- Architectural design can benefit from using architectural patterns, which are proven and reusable solutions to common problems in software design. Architectural patterns provide a common vocabulary and a set of best practices for designing software systems. Some examples of architectural patterns are:
 - Layered pattern: organizes the system into layers of abstraction, each with a specific responsibility and level of detail
 - Client-server pattern: divides the system into clients that request services and servers that provide services over a network
 - Peer-to-peer pattern: distributes the system among peers that can act as both clients and servers
 - Model-view-controller pattern: separates the system into three components: model that manages the data, view that displays the data, and controller that handles the user input
 - Microservices pattern: decomposes the system into small, independent, and loosely coupled services that communicate via messages .

Low Level Design in Software Design

Low level design (LLD) is a component-level design process that follows a step-by-step refinement process. This process can be used for designing data structures, required software architecture, source code and ultimately, performance algorithms .

Some of the benefits of low level design are:

- It helps to ensure the consistency and completeness of the software design.
- It helps to identify and resolve any design issues or errors at an early stage.
- It helps to improve the quality, efficiency and maintainability of the software.
- It helps to facilitate the communication and collaboration among the developers, testers and other stakeholders.

Some of the steps involved in low level design are:

- Identify the modules and components of the software system and their relationships.
- Define the interfaces and inputs/outputs of each module and component.
- Specify the data structures, algorithms and logic of each module and component.
- Document the low level design using appropriate diagrams, notations and tools.

Some of the types of low level design are:

- Object-oriented design: It uses the principles of object-oriented programming to model the software system as a collection of objects that interact with each other through messages .
- Procedural design: It uses the principles of structured programming to model the software system as a sequence of procedures or functions that manipulate data and control the flow of execution.
- Data flow design: It uses the principles of functional programming to model the software system as a network of data transformations that process inputs and produce outputs.

Some of the examples of low level design are:

- Class diagrams: They show the classes, attributes, methods and associations of the software system .
- Sequence diagrams: They show the interactions and messages among the objects of the software system over time .
- Flowcharts: They show the steps, decisions and loops of the software system using symbols and arrows.
- Pseudocode: They show the logic and syntax of the software system using a simplified and readable form of code .

Modularization in Software Design

- Modularization is the process of dividing a software system into smaller, independent, and cohesive modules that can be developed, tested, and maintained separately.
- Modularization has several benefits for software design, such as:
 - Improving readability and understandability of the code by reducing complexity and hiding implementation details.
 - Enhancing reusability and maintainability of the code by allowing modules to be reused in different contexts and modified without affecting other modules.
 - Facilitating parallel development and testing by enabling different teams or developers to work on different modules independently and integrate them later.
 - Increasing reliability and quality of the software by isolating errors and faults within modules and reducing the impact of changes on the overall system.
- Modularization can be achieved by applying various principles and techniques, such as:
 - Abstraction: defining the essential features and behavior of a module without exposing its internal details or dependencies.
 - Encapsulation: hiding the internal data and operations of a module from other modules and providing a well-defined interface for communication and interaction.
 - Cohesion: ensuring that a module has a single, clear, and consistent purpose and responsibility and that its elements are strongly related and relevant to each other.
 - Coupling: minimizing the degree of interdependence and interaction between modules and avoiding unnecessary or excessive dependencies and communications.
 - Decomposition: breaking down a complex problem or system into smaller and simpler subproblems or subsystems that can be solved or implemented as modules.
 - Composition: combining and integrating modules to form a larger and more complex system that provides the desired functionality and behavior.

Design Structure Charts in Software Design

- A design structure chart (DSC) is a diagram that shows the hierarchical decomposition of a software system into its modules and submodules.
- A DSC is useful for representing the logical structure and functional relationships of a software system, as well as the data flow and control flow between the modules.
- A DSC can help in the design, implementation, testing, and maintenance of a software system, by providing a clear and consistent view of its components and their interactions.
- A DSC consists of the following elements:

- Modules: Rectangular boxes that represent the basic units of functionality in the software system. Each module has a name and a number that indicates its level in the hierarchy. The top-level module is numbered 0, and the submodules are numbered 1, 2, 3, etc. A module can have zero or more submodules, and a submodule can belong to only one module.
- Connectors: Lines that connect the modules and show the direction of data flow and control flow between them. A connector can have one of the following types:
 - * Call: A solid line with an arrowhead that indicates a module calls another module and passes some data or parameters to it. The arrow points from the caller to the callee.
 - * Return: A dashed line with an arrowhead that indicates a module returns some data or results to another module. The arrow points from the callee to the caller.
 - * Jump: A dotted line with an arrowhead that indicates a module transfers the control flow to another module without passing any data or parameters. The arrow points from the source to the destination.
- Data: Labels that show the name and type of the data or parameters that are passed or returned between the modules. The data labels are placed near the connectors that carry them.
- Conditions: Labels that show the logical conditions or expressions that determine the control flow between the modules. The conditions are placed near the connectors that depend on them.
- Fan-in: The number of incoming connectors to a module, which indicates how many modules call or use that module.
- Fan-out: The number of outgoing connectors from a module, which indicates how many modules are called or used by that module.
- Coupling: The degree of interdependence or interaction between the modules, which depends on the number and type of connectors and data between them. Low coupling is desirable, as it means the modules are more independent and easier to modify and reuse.
- Cohesion: The degree of relatedness or similarity of the functionality within a module, which depends on the number and type of submodules and data within it. High cohesion is desirable, as it means the module is more focused and easier to understand and test.
- A DSC can be drawn using various tools or notations, such as Yourdon/Constantine, Warnier/Orr, or Nassi/Shneiderman. The following is an example of a DSC for a simple calculator software system, using the Yourdon/Constantine notation:

DSC example

- The DSC shows that the calculator system consists of four modules: Main (0), Input (1), Calculate (2), and Output (3). The Main module calls the Input module to get the operands and the operator from the user, then

calls the Calculate module to perform the arithmetic operation, and then calls the Output module to display the result. The Input, Calculate, and Output modules return the data to the Main module. The Calculate module has four submodules: Add (2.1), Subtract (2.2), Multiply (2.3), and Divide (2.4), which perform the corresponding operations. The Calculate module uses a condition to decide which submodule to call, based on the operator. The data labels show the names and types of the data passed or returned between the modules, such as op1, op2, op, and res. The fan-in and fan-out numbers show the number of incoming and outgoing connectors for each module. The coupling and cohesion of the system can be assessed by analyzing the connectors and data between and within the modules.

Pseudo Codes in Software Design

- Pseudo code is a term which is often used in programming and algorithm based fields. It is a methodology that allows the programmer to represent the implementation of an algorithm.
- Pseudo code is an important part of designing an algorithm, it helps the programmer in planning the solution to the problem as well as the reader in understanding the approach to the problem .
- Pseudo code is an intermediate state between algorithm and program that plays supports the transition of the algorithm into the program .
- Pseudo code is not a formal language, it does not have any syntax or grammar rules. It is written in a natural language that is easy to read and understand .
- Pseudo code uses keywords, symbols, and indentation to structure the logic and flow of the algorithm .
- Pseudo code can be written at different levels of abstraction, depending on the purpose and audience of the pseudo code .
- Pseudo code can be used for various purposes, such as:
 - Designing code in a group setting. High-level software architects will often include pseudo code in their design process to help solve a complex problem they see their programmers running into.
 - Testing and debugging code. Pseudo code can help the programmer to identify and fix errors in the logic and flow of the algorithm before writing the actual code .
 - Documenting code. Pseudo code can serve as a concise and clear explanation of the algorithm and its functionality .
 - Learning and teaching code. Pseudo code can help beginners to understand the basic concepts and structures of programming, as well as to practice their problem-solving skills .
- Pseudo code can be written in various formats and styles, depending on the preference and convention of the programmer. However, some general guidelines for writing pseudo code are :
 - Use a consistent indentation to show the hierarchy and nesting of the

statements.

- Use capital letters for keywords, such as IF, ELSE, WHILE, FOR, etc.
- Use meaningful and descriptive names for variables, functions, and parameters.
- Use comments to explain the purpose and logic of the pseudo code.
- Use simple and clear language to express the algorithm.
- Use pseudocode to solve coding problems.

Flow Charts in Software Design

- A flow chart is a graphical or symbolic representation of a process or an algorithm that shows the steps, decisions, and data flows involved in solving a problem or achieving a goal .
- Flow charts are useful in software design because they can help in planning, documenting, testing, debugging, and communicating the logic and structure of a program or an algorithm .
- Flow charts use standard symbols and shapes to represent different types of actions, inputs, outputs, and conditions in a process or an algorithm . Some of the common symbols are:

Symbol	Meaning
Oval	Start or end of a process or an algorithm
Rectangle	A step or an action to be performed
Diamond	A decision or a branching point with two or more possible outcomes
Parallelogram	An input or an output of data or information
Arrow	A direction or a flow of control

- Flow charts can be drawn using various software tools, such as Lucidchart, Textografo, Visio, etc. that offer features like drag-and-drop, text-based, or template-based flowchart creation .
- Flow charts can be used to design different types of software processes or algorithms, such as:
 - Sequential processes or algorithms that follow a linear order of steps or actions, such as calculating the factorial of a number, sorting an array, etc.
 - Conditional processes or algorithms that involve branching or looping based on certain conditions or criteria, such as finding the maximum element in an array, checking if a number is prime, etc.
 - Parallel processes or algorithms that involve concurrent or simultaneous execution of multiple steps or actions, such as multithreading,

multiprocessing, etc.

Coupling in Software Design

- Coupling is the degree of interdependence between software modules; a measure of how closely connected two routines or modules are; the strength of the relationships between modules.
- Coupling is usually contrasted with cohesion, which is the degree of intra-dependence within a module; a measure of how well the elements of a module belong together.
- Low coupling often correlates with high cohesion, and vice versa. Low coupling means that modules are independent and changes in one module have little impact on other modules. High cohesion means that modules are focused and have a single responsibility.
- Coupling is an important factor in software design, as it affects the quality, maintainability, reusability, and testability of the software. High coupling increases the complexity and brittleness of the software, making it harder to understand, modify, and test. Low coupling reduces the complexity and increases the modularity and flexibility of the software, making it easier to understand, modify, and test.
- There are different types of coupling, depending on the nature and degree of the dependencies between modules. Some common types of coupling are:
 - Common coupling: when modules share global data or constants.
 - Content coupling: when modules directly access or modify the contents of another module.
 - Data coupling: when modules communicate through parameters or return values.
 - Stamp coupling: when modules communicate through data structures or objects.
 - Control coupling: when modules communicate through control flags or signals.
 - External coupling: when modules depend on external systems or interfaces.
- The types of coupling can be arranged in a hierarchy, from the strongest to the weakest, as follows:
 - Content coupling > Common coupling > External coupling > Control coupling > Stamp coupling > Data coupling > No coupling
- The goal of software design is to achieve low coupling and high cohesion, as this improves the quality and maintainability of the software. Some techniques to achieve low coupling are:
 - Encapsulation: hiding the implementation details of a module and exposing only its interface.
 - Abstraction: providing a general and simplified representation of a module and its functionality.
 - Modularity: dividing the software into smaller and independent units

that can be composed together.

- Information hiding: concealing the details of a module that are not relevant to other modules.
- Loose coupling: minimizing the dependencies between modules and using interfaces or contracts to communicate.

Cohesion Measures in Software Design

- Cohesion is a measure of how strongly related and focused the various responsibilities of a software module are.
- A software module can be a class, a package, a component, a subsystem or a system.
- A software module has high cohesion if all its elements (such as instructions, data, functions, etc.) are closely related to each other and to the main purpose of the module.
- A software module has low cohesion if its elements are loosely related or unrelated to each other or to the main purpose of the module.
- High cohesion is desirable in software design because it is associated with several benefits, such as :
 - Robustness: High cohesion reduces the chances of errors and bugs in the module, as the elements are well-defined and consistent.
 - Reliability: High cohesion increases the confidence in the correctness and functionality of the module, as the elements are well-tested and verified.
 - Reusability: High cohesion makes the module more suitable for reuse in different contexts, as the elements are self-contained and independent.
 - Understandability: High cohesion makes the module easier to understand and maintain, as the elements are clear and coherent.
- Low cohesion is undesirable in software design because it is associated with several drawbacks, such as :
 - Complexity: Low cohesion increases the complexity and size of the module, as the elements are diverse and scattered.
 - Coupling: Low cohesion increases the dependency and interaction between the module and other modules, as the elements are not isolated and encapsulated.
 - Modifiability: Low cohesion makes the module harder to modify and extend, as the elements are interdependent and inconsistent.
 - Testability: Low cohesion makes the module harder to test and debug, as the elements are ambiguous and unpredictable.
- Cohesion is an ordinal type of measurement and is usually described as “high cohesion” or “low cohesion”. However, there are also different levels or types of cohesion that can be used to classify and compare software modules, such as:
 - Functional cohesion: The highest level of cohesion, where the module performs a single and specific function.

- Sequential cohesion: The second highest level of cohesion, where the module performs a series of related actions that follow a logical sequence.
- Communicational cohesion: The third highest level of cohesion, where the module performs a set of related actions that operate on the same data or input/output.
- Procedural cohesion: The fourth highest level of cohesion, where the module performs a set of related actions that are grouped together by control flow or procedure.
- Temporal cohesion: The fifth highest level of cohesion, where the module performs a set of related actions that are grouped together by time or event.
- Logical cohesion: The sixth highest level of cohesion, where the module performs a set of related actions that are grouped together by logic or condition.
- Coincidental cohesion: The lowest level of cohesion, where the module performs a set of unrelated or arbitrary actions that are grouped together by coincidence or convenience.
- Cohesion can be measured using different metrics or techniques, such as:
 - Slice-based cohesion: A metric that measures the cohesion of a module based on the number and size of its executable slices, which are subsets of statements that affect or are affected by a given variable.
 - Information-flow-based cohesion: A metric that measures the cohesion of a module based on the amount and direction of information flow between its elements, such as data, control, and invocation.
 - Semantic cohesion: A metric that measures the cohesion of a module based on the similarity of meaning or purpose of its elements, such as keywords, identifiers, and comments.
 - Structural cohesion: A metric that measures the cohesion of a module based on the structure and organization of its elements, such as hierarchy, modularity, and encapsulation.

Design Strategies in Software Design

- Design strategies are the approaches that are taken to design a software system that meets the requirements and specifications of the problem domain.
- Design strategies aim to achieve a balance between the quality attributes of the software, such as performance, reliability, usability, security, etc.
- Design strategies can be classified into three major categories: structured design, function-oriented design, and object-oriented design.
 - Structured design is a design strategy that focuses on the decomposition of the system into modules that have well-defined interfaces and responsibilities. Structured design follows a top-down approach, where the system is divided into sub-systems, and then into smaller components, until the desired level of abstraction is reached. Struc-

tured design is suitable for simple and stable systems that do not require frequent changes or extensions.

- Function-oriented design is a design strategy that focuses on the identification and grouping of the functions or operations that the system performs. Function-oriented design follows a bottom-up approach, where the system is built from the basic functions, and then integrated into higher-level functions, until the desired functionality is achieved. Function-oriented design is suitable for complex and dynamic systems that require high performance and efficiency.
- Object-oriented design is a design strategy that focuses on the identification and grouping of the objects or entities that the system interacts with. Object-oriented design follows a hybrid approach, where the system is modeled as a collection of objects that have attributes, behaviors, and relationships. Object-oriented design is suitable for modular and flexible systems that require reusability and extensibility.

Function Oriented Design in Software Design

- Function Oriented Design is an approach to software design where the design is decomposed into a set of interacting units or modules where each unit or module has a clearly defined function .
- The main focus of Function Oriented Design is on the functions or operations that the software performs, rather than the data or objects that it manipulates.
- Function Oriented Design is based on the principle of top-down decomposition, where the system is divided into smaller and simpler sub-systems, and each sub-system is further divided into smaller and simpler modules, until the modules are simple enough to be implemented.
- Function Oriented Design uses various notations and diagrams to represent the functional structure and behavior of the software, such as:
 - Data Flow Diagrams (DFDs): DFDs show the flow of data and control among the modules and external entities .
 - Data Dictionaries: Data dictionaries store information about the data items defined in the DFDs, such as their names, types, formats, sources, destinations, etc.
 - Structure Charts: Structure Charts show the hierarchical relationship and communication among the modules .
 - Pseudo-code: Pseudo-code is a textual representation of the algorithm or logic of a module.
- Function Oriented Design has some advantages, such as:
 - It is easy to understand and communicate, as it uses natural language and graphical notations.
 - It is suitable for small and simple systems that have well-defined functions and data flows.
 - It supports modularization and reusability of code, as the modules

- can be tested and maintained independently.
- Function Oriented Design also has some disadvantages, such as:
 - It does not capture the data abstraction and encapsulation aspects of the software, as it does not focus on the data or objects that the software manipulates.
 - It does not support inheritance and polymorphism, which are the key features of object-oriented design.
 - It may lead to poor design quality and performance, as the modules may have high coupling and low cohesion, which means that they depend on each other too much and do not perform a single function well.

Object Oriented Design in Software Design

- Object oriented design (OOD) is the process of using an object oriented methodology to design a computing system or application .
- OOD enables the implementation of a software solution based on the concepts of objects, which are entities that have attributes (data) and behaviors (methods) .
- OOD serves as part of the object oriented programming (OOP) process or lifecycle, which includes analysis, design, implementation, testing and maintenance .
- OOD aims to create software that is modular, reusable, extensible, maintainable and adaptable .
- OOD follows some principles and guidelines, such as:
 - Abstraction: hiding the details and complexity of a system and exposing only the essential features .
 - Encapsulation: bundling the data and methods of an object and restricting access to them from outside .
 - Inheritance: allowing a subclass to inherit the attributes and methods of a superclass and add new ones .
 - Polymorphism: allowing an object to have different forms or behaviors depending on the context .
 - SOLID: a set of five principles that promote good OOD practices, such as single responsibility, open-closed, Liskov substitution, interface segregation and dependency inversion .
- OOD can be applied using various tools and techniques, such as:
 - UML: a standard graphical notation for modeling the structure and behavior of a system using diagrams .
 - Design patterns: reusable solutions to common design problems that describe the relationships and interactions among objects .
 - Refactoring: improving the design of existing code by changing its structure without altering its functionality .
 - Testing: verifying the correctness and quality of the software using various methods, such as unit testing, integration testing and system testing .

Top-Down and Bottom-Up Design in Software Design

- Top-down and bottom-up are two approaches for designing software systems.
- Top-down design starts with a high-level overview of the system and decomposes it into smaller and more specific components. The components are then designed and implemented in detail, and integrated to form the complete system.
- Bottom-up design starts with the low-level components of the system and builds them up into larger and more abstract modules. The modules are then combined and tested to form the complete system.
- Both approaches have advantages and disadvantages, and can be used in different situations and contexts.
- Some of the advantages of top-down design are:
 - It helps to clarify the overall goals and requirements of the system.
 - It facilitates the division of work among different developers or teams.
 - It allows for early testing and verification of the system functionality and performance.
- Some of the disadvantages of top-down design are:
 - It may lead to incomplete or inaccurate specifications of the lower-level components.
 - It may cause delays or difficulties in integrating the components due to interface or compatibility issues.
 - It may require more changes or revisions as the system evolves or new requirements emerge.
- Some of the advantages of bottom-up design are:
 - It allows for the reuse of existing or standard components or libraries.
 - It enables the parallel development and testing of the components.
 - It reduces the risk of errors or failures in the system as the components are already tested and verified.
- Some of the disadvantages of bottom-up design are:
 - It may result in a lack of coherence or consistency in the system architecture or design.
 - It may require more effort or coordination to ensure the compatibility and integration of the components.
 - It may overlook or neglect some of the high-level goals or requirements of the system.

Software Measurement and Metrics in Software Design

- Software measurement and metrics are used to assess the quality, complexity, reliability, performance, usability and maintainability of software products and processes.
- Software measurement is the process of collecting data on the software and its specifications. Software metrics are the functions that assign a numerical value to an attribute of a software product or process.

- Software measurement and metrics can help software engineers and developers to communicate the status of software development projects, identify and address issues, monitor and improve their workflow, and plan and estimate future work items.
- There are three types of software metrics: product metrics, process metrics and project metrics.
 - Product metrics are used to evaluate the state of the software product, such as its size, complexity, functionality, quality, reliability, usability and maintainability. Examples of product metrics are lines of code, cyclomatic complexity, defect density, mean time to failure, user satisfaction and technical debt.
 - Process metrics are used to monitor and improve the long-term software development process, such as its efficiency, effectiveness, productivity, quality and maturity. Examples of process metrics are defect removal efficiency, defect leakage, process capability, process performance index and return on investment.
 - Project metrics are used to control and manage the software development project, such as its scope, schedule, cost, risk and resources. Examples of project metrics are earned value, schedule variance, cost variance, risk exposure and resource utilization.

Various Size Oriented Measures in Software Design

- Size oriented measures are based on the assumption that the size of the software product is an indicator of its complexity, effort, and quality.
- Size oriented measures can be classified into two categories: direct measures and indirect measures.
- Direct measures are based on the physical attributes of the software product, such as lines of code, number of statements, number of modules, etc.
- Indirect measures are based on the logical attributes of the software product, such as functionality, requirements, design, etc.
- Some examples of direct measures are:
 - Source Lines of Code (SLOC): the number of lines of code in the source program, excluding comments and blank lines.
 - Statement Count: the number of executable statements in the source program, excluding declarations and comments.
 - Function Point (FP): a measure of the functionality delivered by the software product, based on the number and complexity of inputs, outputs, inquiries, files, and interfaces.
- Some examples of indirect measures are:
 - Use Case Point (UCP): a measure of the functionality delivered by the software product, based on the number and complexity of use cases and actors.
 - Feature Point (FEP): a measure of the functionality delivered by the software product, based on the number and complexity of features.
 - Object Point (OP): a measure of the functionality delivered by the

software product, based on the number and complexity of classes, methods, and attributes.

- Size oriented measures can be used for various purposes, such as estimating effort, cost, and schedule, evaluating productivity and quality, and comparing different software products or processes.

Halestead's Software Science in software design

- Halestead's Software Science is a set of software metrics that aim to measure the complexity, quality, and effort of a program based on its source code .
- The premise of Halestead's Software Science is that any programming task consists of selecting and arranging a finite number of program tokens, which are basic syntactic units distinguishable by a compiler.
- Halestead's Software Science defines two types of tokens: operators and operands. Operators are symbols that represent actions or functions, such as +, =, if, while, etc. Operands are symbols that represent data or values, such as variables, constants, literals, etc .
- Halestead's Software Science uses four primitive program parameters to derive the expressions for various software metrics :
 - n1: the number of distinct operators in the program
 - n2: the number of distinct operands in the program
 - N1: the total number of operators in the program
 - N2: the total number of operands in the program
- Some of the software metrics that Halestead's Software Science provides are :
 - Program length (N): the total number of tokens in the program, $N = N1 + N2$
 - Program vocabulary (n): the number of distinct tokens in the program, $n = n1 + n2$
 - Volume (V): the amount of information contained in the program, $V = N * \log_2(n)$
 - Difficulty (D): the difficulty of writing or understanding the program, $D = (n1/2) * (N2/n2)$
 - Effort (E): the amount of mental effort required to develop the program, $E = D * V$
 - Time (T): the estimated time required to develop the program, $T = E / 18 \text{ seconds}$
 - Bugs (B): the estimated number of errors in the program, $B = V / 3000$
- Halestead's Software Science has some advantages and disadvantages as a software metric system :

- Advantages:
 - * It is easy to automate and apply to any programming language
 - * It is based on objective and measurable properties of the source code
 - * It can provide some insights into the complexity and quality of the program
- Disadvantages:
 - * It does not account for the semantic aspects of the program, such as logic, design, or functionality
 - * It does not consider the external factors that affect the development process, such as requirements, testing, or maintenance
 - * It is based on some assumptions and empirical constants that may not be valid or generalizable for all programs

Function Point (FP) Based Measures in software design

- Function Point (FP) is a unit of measure for software functionality that reflects the user's perspective and requirements .
- FP is used to estimate the cost, duration, and resources needed for software development and maintenance .
- FP is independent of the technology, programming language, or methodology used for implementation .
- FP is calculated based on the number and complexity of various components of the software, such as inputs, outputs, inquiries, files, and interfaces .
- FP is adjusted by a Complexity Adjustment Factor (CAF) that accounts for the influence of 14 general system characteristics, such as data communications, performance, reusability, etc .
- FP can be measured at different stages of the software life cycle, such as requirements, design, or testing .
- FP can be used to compare the productivity, quality, and efficiency of different software projects or processes .

Cyclomatic Complexity Measures in Software Design

- Cyclomatic complexity is a software metric used to indicate the complexity of a program .
- It is a quantitative measure of the number of linearly independent paths through a program's source code .
- It was developed by Thomas J. McCabe, Sr. in 1976 .
- It can be calculated using the following formula :

$$- M = E - N + 2P$$

- where M is the cyclomatic complexity, E is the number of edges, N is the number of nodes, and P is the number of connected components in the control flow graph of the program.
- Cyclomatic complexity can be used for the following purposes :
 - Determining the independent path executions thus proven to be very helpful for developers and testers .
 - Estimating the minimum number of test cases required to cover all the paths of the program .
 - Evaluating the maintainability and readability of the code .
 - Identifying the potential errors and bugs in the code .
 - Refactoring the code to reduce the complexity and improve the quality .
- Cyclomatic complexity can be measured using various tools, such as Visual Studio, CodeSonar, CodeScene, etc.

Control Flow Graphs in software design

- A control flow graph (CFG) is a graphical representation of the possible paths of execution of a program or a function.
- A CFG consists of nodes and edges, where each node represents a basic block of code (a sequence of statements that are always executed together) and each edge represents a possible transfer of control between basic blocks.
- A CFG has a single entry node, where the execution starts, and one or more exit nodes, where the execution ends. The entry node has no incoming edges and the exit nodes have no outgoing edges.
- A CFG can be used to analyze various properties of a program or a function, such as reachability, liveness, dominance, loop detection, data flow, etc.
- A CFG can also be used to optimize a program or a function, such as eliminating dead code, performing constant propagation, applying loop transformations, etc.
- A CFG can be constructed from the source code or the intermediate code of a program or a function, using various algorithms, such as the one proposed by Allen and Cocke (1971).
- A CFG can be represented in various ways, such as using a graphical notation, a textual notation, or a matrix notation.
- A CFG can be modified to include additional information, such as annotations, labels, weights, etc., depending on the purpose of the analysis or the optimization.
- A CFG can be combined with other CFGs to form a call graph, which represents the interprocedural control flow of a program. A call graph can be used to perform interprocedural analysis or optimization, such as inlining, tail recursion elimination, etc.
- A CFG can be transformed into other forms of representation, such as

a data flow graph, a program dependence graph, a control dependence graph, etc., to facilitate different kinds of analysis or optimization.

Unit 4 - Software Testing

Software testing is the process of verifying and validating that a software product meets the requirements and expectations of the stakeholders. Software testing can be performed at different levels of development, such as unit testing, integration testing, system testing, and acceptance testing. Software testing can also be classified into different types, such as functional testing, non-functional testing, white-box testing, black-box testing, and grey-box testing. Software testing can be done manually or with the help of automated tools.

Some of the objectives of software testing are:

- To ensure that the software product conforms to the specifications and requirements.
- To identify and eliminate defects and errors in the software product.
- To improve the quality and reliability of the software product.
- To reduce the risk of failure and ensure customer satisfaction.
- To provide feedback and information for improvement and maintenance.

Some of the principles of software testing are:

- Testing shows the presence of defects, not the absence of defects.
- Exhaustive testing is impossible, so testing should be based on priorities and risks.
- Early testing saves time and cost, so testing should start as soon as possible in the development cycle.
- Defects tend to cluster, so testing should focus on the areas with high defect density.
- Testing is context-dependent, so testing should be tailored to the specific software product and its intended use.
- Testing is a creative and intellectual activity, so testing should involve human skills and judgment.

Testing Objectives in Software Testing

- Testing objectives are the goals and expectations that guide the testing process and its outcomes.
- One of the main objectives of software testing is to **improve software quality** by finding and fixing defects before they reach the end users.
- Another objective of software testing is to **gain confidence** in and provide information about the level of quality and reliability of the software.
- A third objective of software testing is to **verify** whether the software meets the specified requirements and expectations of the stakeholders.
- A fourth objective of software testing is to **prevent** defects from occurring in the first place by applying best practices and standards throughout the

software development life cycle.

- Testing objectives may vary depending on the type of testing technique and the software development model used. For example, unit testing aims to find as many defects as possible in individual components, while integration testing focuses on the interactions between components.
- Testing objectives should be **clear**, **measurable**, **realistic**, and **aligned** with the project goals and scope. They should also be communicated and agreed upon by all the stakeholders involved in the testing process.

Unit Testing in Software Testing

- Unit testing is a type of software testing that focuses on verifying the functionality and quality of individual units of code, such as functions, methods, classes, or modules.
- Unit testing is usually performed by the developers who write the code, using tools and frameworks that automate the execution and verification of test cases.
- Unit testing helps to ensure that each unit of code works as expected, and to detect and fix errors or bugs at an early stage of development.
- Unit testing also facilitates code refactoring, debugging, integration, and maintenance, by providing a reliable and fast way to verify the correctness of the code.
- Unit testing follows the principle of “test early, test often”, and is typically integrated into the software development process as part of an agile or iterative methodology.
- Unit testing can be done in various ways, such as using assertions, mock objects, stubs, test doubles, or code coverage tools.
- Unit testing can be classified into two categories: black-box testing and white-box testing.
 - Black-box testing treats the unit of code as a black box, and only tests its inputs and outputs, without knowing its internal structure or logic.
 - White-box testing examines the unit of code as a white box, and tests its internal structure, logic, and branches, using the knowledge of the code implementation.
- Unit testing can be further divided into positive testing and negative testing.
 - Positive testing verifies that the unit of code behaves correctly when given valid inputs and expected scenarios.
 - Negative testing checks that the unit of code handles invalid inputs, errors, exceptions, and unexpected scenarios gracefully.

Integration Testing in Software Testing

- Integration testing is a level of software testing that involves testing the interactions and interfaces between different components or modules of a

software system.

- Integration testing aims to verify the functionality, performance, reliability, and compatibility of the integrated components or modules.
- Integration testing can be performed using different approaches, such as:
 - **Big-bang integration testing:** All the components or modules are integrated and tested together as a whole. This approach is suitable for small and simple systems, but it can be risky and time-consuming for large and complex systems.
 - **Incremental integration testing:** The components or modules are integrated and tested gradually, in a bottom-up, top-down, or sandwich manner. This approach is more manageable and efficient for large and complex systems, but it requires more planning and coordination.
 - **Hybrid integration testing:** A combination of big-bang and incremental integration testing approaches. This approach can balance the advantages and disadvantages of both approaches, depending on the system requirements and characteristics.
- Integration testing can be performed using different techniques, such as:
 - **Stub:** A dummy component or module that simulates the behavior of a missing or incomplete component or module. Stubs are used to support bottom-up integration testing, where the lower-level components or modules are tested first.
 - **Driver:** A dummy component or module that invokes and tests the behavior of a component or module under test. Drivers are used to support top-down integration testing, where the higher-level components or modules are tested first.
 - **Test harness:** A collection of tools and software that facilitate the execution and evaluation of integration testing. Test harnesses can include stubs, drivers, test cases, test data, test scripts, test reports, etc.
- Integration testing can be performed using different types of tools, such as:
 - **Static analysis tools:** Tools that analyze the source code or design of the components or modules to identify potential errors, defects, or inconsistencies in the integration. Static analysis tools can include code analyzers, code reviewers, code metrics, etc.
 - **Dynamic analysis tools:** Tools that execute the components or modules and monitor their behavior, performance, or output during the integration. Dynamic analysis tools can include debuggers, profilers, tracers, monitors, etc.
 - **Test automation tools:** Tools that automate the creation, execution, and evaluation of integration testing. Test automation tools can include test frameworks, test libraries, test generators, test runners, etc.

Acceptance Testing in Software Testing

- Acceptance testing is one of the levels of software testing that evaluates the system's compliance with the business requirements and user needs.
- The purpose of acceptance testing is to determine whether the system satisfies the acceptance criteria and to enable the user, customers or other authorized entity to decide whether to accept the system.
- Acceptance testing is performed after system testing and before making the system available for actual use.
- Acceptance testing can be done by the end-users, clients, or other stakeholders with the help of test cases or user stories.
- Acceptance testing can be classified into different types, such as:
 - User acceptance testing (UAT): User acceptance testing is used to verify whether the product is working for the user correctly and meets their expectations.
 - Business acceptance testing (BAT): Business acceptance testing is used to check whether the product aligns with the business goals and objectives.
 - Contract acceptance testing (CAT): Contract acceptance testing is used to validate whether the product meets the contractual agreements and specifications.
 - Regulation acceptance testing (RAT): Regulation acceptance testing is used to ensure that the product complies with the legal and regulatory standards.
 - Operational acceptance testing (OAT): Operational acceptance testing is used to evaluate whether the product is ready for deployment and operation in the real environment.
- Acceptance testing can be conducted in different ways, such as:
 - Alpha testing: Alpha testing is done by the internal team of developers and testers at the development site before releasing the product to the external users.
 - Beta testing: Beta testing is done by a limited number of external users or customers at their own site after releasing the product to them.
 - Gamma testing: Gamma testing is done by the end-users or customers at their own site after the product is fully released to the market.
- Acceptance testing helps in finding out the efficiency of development and testing teams, improving the quality of the product, increasing the customer satisfaction, and reducing the risks of failure and maintenance costs.

Regression Testing in Software Testing

- Regression testing is the process of verifying that a software system or application does not have any new or existing defects after a change or update is made to it.

- Regression testing is important to ensure that the software quality and functionality are maintained and improved after any modification, such as bug fixes, enhancements, patches, configuration changes, etc.
- Regression testing can be performed at different levels of testing, such as unit testing, integration testing, system testing, and acceptance testing, depending on the scope and impact of the change.
- Regression testing can be done manually or automatically, depending on the availability and suitability of test cases, test tools, test resources, and test environment.
- Regression testing can be classified into different types, such as:
 - Retest all: This type of regression testing involves re-executing all the existing test cases for the entire software system or application, regardless of the change. This type of regression testing is comprehensive, but time-consuming and costly.
 - Selective: This type of regression testing involves selecting and re-executing only a subset of the existing test cases that are relevant to the change. This type of regression testing is efficient, but requires a proper test case selection strategy and criteria.
 - Prioritized: This type of regression testing involves prioritizing and re-executing the existing test cases based on some factors, such as risk, impact, complexity, frequency, etc. This type of regression testing is effective, but requires a proper test case prioritization technique and metric.
 - Progressive: This type of regression testing involves adding and executing new test cases for the new or modified functionality, along with re-executing some of the existing test cases. This type of regression testing is adaptive, but requires a proper test case design and maintenance process.
 - Retest partial: This type of regression testing involves re-executing only a part of the existing test cases for the modified functionality, instead of the whole functionality. This type of regression testing is fast, but requires a proper test case coverage and traceability analysis.

Testing for Functionality in Software Testing

- Functionality testing is a type of software testing that verifies that the software performs as expected and meets the functional requirements specified by the customer.
- Functionality testing covers various aspects of software functionality, such as usability, accessibility, compatibility, security, performance, reliability, and compliance.
- Functionality testing can be performed at different levels of software development, such as unit testing, integration testing, system testing, and acceptance testing.

- Functionality testing can be done manually or with the help of automated tools, depending on the complexity and scope of the software.
- Functionality testing can be done using various techniques, such as black-box testing, white-box testing, gray-box testing, exploratory testing, and scenario-based testing.
- Functionality testing can be done using various methods, such as equivalence partitioning, boundary value analysis, decision table testing, state transition testing, use case testing, and user story testing.
- Functionality testing can be done using various types of test cases, such as positive test cases, negative test cases, error test cases, and boundary test cases.
- Functionality testing can be done using various metrics, such as test coverage, defect density, defect severity, defect priority, and defect resolution time.
- Functionality testing can be done using various standards, such as IEEE 829, ISO 9126, ISO 25010, and ISO 29119.
- Functionality testing can be done using various best practices, such as defining clear and testable requirements, designing test cases based on requirements, executing test cases systematically, reporting and tracking defects, and performing regression testing.

Testing for Performance in Software Testing

- Performance testing is a type of software testing that aims to evaluate the quality attributes of a system related to speed, scalability, reliability, and resource consumption.
- Performance testing can help to identify and eliminate performance bottlenecks, optimize the system configuration, and ensure that the system meets the specified performance requirements and expectations.
- Performance testing can be classified into different types, such as load testing, stress testing, endurance testing, spike testing, volume testing, and scalability testing.
- Load testing is the process of applying a realistic or expected load on the system and measuring its response time, throughput, and resource utilization. Load testing can help to determine the maximum capacity of the system and its behavior under normal and peak conditions.
- Stress testing is the process of applying an extreme or abnormal load on the system and observing its behavior and stability. Stress testing can help to identify the breaking point of the system and its failure modes.
- Endurance testing is the process of applying a continuous or sustained load on the system for a long period of time and monitoring its performance and reliability. Endurance testing can help to detect memory leaks, resource leaks, and degradation issues.
- Spike testing is the process of applying a sudden and unexpected increase or decrease in the load on the system and measuring its ability to handle the fluctuations. Spike testing can help to evaluate the robustness and

resilience of the system.

- Volume testing is the process of applying a large amount of data on the system and assessing its performance and throughput. Volume testing can help to verify the data handling capacity of the system and its components.
- Scalability testing is the process of increasing or decreasing the load, data, or resources on the system and evaluating its performance and scalability. Scalability testing can help to determine the optimal configuration of the system and its scalability limits.

Top-Down and Bottom-Up Testing Strategies in Software Testing

- Top-down and bottom-up are two approaches to testing the integration of software components or modules.
- Top-down testing starts with the highest-level module and gradually integrates the lower-level modules using stubs. Stubs are dummy modules that simulate the behavior of the actual modules that are not yet integrated.
- Bottom-up testing starts with the lowest-level modules and gradually integrates the higher-level modules using drivers. Drivers are test modules that invoke and pass test data to the modules that are being integrated.
- The advantages of top-down testing are:
 - It allows early testing of the main functionality and critical features of the software.
 - It helps to identify and fix major design flaws and interface errors at an early stage.
 - It facilitates top-level management and control of the testing process.
- The disadvantages of top-down testing are:
 - It requires a lot of stubs to be developed and maintained, which can be time-consuming and complex.
 - It may delay the testing of some functionalities and features that depend on the lower-level modules.
 - It may not detect some errors or bugs that occur only in the interaction of the lower-level modules.
- The advantages of bottom-up testing are:
 - It allows early testing of the basic functionalities and features of the software.
 - It helps to identify and fix low-level errors and bugs at an early stage.
 - It reduces the need for stubs and simplifies the testing process.
- The disadvantages of bottom-up testing are:
 - It requires a lot of drivers to be developed and maintained, which can be time-consuming and complex.
 - It may delay the testing of some functionalities and features that depend on the higher-level modules.
 - It may not detect some errors or bugs that occur only in the interaction of the higher-level modules.

Test Drivers and Test Stubs software testing strategy

- Test drivers and test stubs are used to facilitate the testing of software components or modules that are not yet integrated or complete.
- Test drivers are programs that simulate the behavior of a module that calls another module or component. They provide the input data, invoke the module under test, and display the output results.
- Test stubs are programs that simulate the behavior of a module that is called by another module or component. They accept the input data, perform some minimal processing, and return the expected output data.
- Test drivers and test stubs are useful for bottom-up and top-down integration testing strategies, respectively.
- Bottom-up integration testing is a strategy that starts with testing the lowest level modules or components and gradually integrates and tests the higher level ones until the entire system is tested.
- Top-down integration testing is a strategy that starts with testing the highest level modules or components and gradually integrates and tests the lower level ones until the entire system is tested.
- Test drivers and test stubs help to isolate the module under test from the dependencies and interactions with other modules or components, and allow the tester to focus on the functionality and correctness of the module under test.
- Test drivers and test stubs can be written manually by the tester or developer, or generated automatically by some tools or frameworks.

Structural Testing (White Box Testing) software testing strategy

- Structural testing, also known as white box testing, is a software testing strategy that focuses on the internal structure, design, and implementation of the software system.
- The main objective of structural testing is to verify that the software conforms to the specified design and coding standards, and that it has no errors or defects in its logic, data flow, or control flow.
- Structural testing requires the tester to have access to the source code and the design documents of the software system. The tester can use various techniques and tools to analyze the code and generate test cases that cover different paths, branches, statements, and conditions in the code.
- Some of the common techniques and tools for structural testing are:
 - Code coverage: A measure of how much of the code is executed by the test cases. It can be calculated at different levels, such as statement coverage, branch coverage, condition coverage, path coverage, etc. Code coverage tools can help the tester identify the parts of the code that are not tested or have low coverage, and generate additional test cases to increase the coverage.
 - Control flow testing: A technique that uses the control flow graph of the code to derive test cases that cover all the possible paths from

the entry point to the exit point of the code. Control flow testing can help the tester detect errors or anomalies in the logic or sequence of the code, such as loops, jumps, dead code, etc.

- Data flow testing: A technique that uses the data flow graph of the code to derive test cases that cover all the possible definitions, uses, and kills of the variables in the code. Data flow testing can help the tester detect errors or anomalies in the data or information flow of the code, such as uninitialized variables, dangling pointers, memory leaks, etc.
- Mutation testing: A technique that involves creating and executing variants of the code by introducing small changes or mutations, such as changing operators, operands, statements, etc. Mutation testing can help the tester evaluate the effectiveness of the test cases by measuring how many mutants are detected or killed by the test cases. A high mutation score indicates that the test cases are able to detect most of the faults in the code.

Functional Testing (Black Box Testing) software testing strategy

- Functional testing is a software testing strategy that focuses on verifying the functionality of the software according to the specified requirements.
- Functional testing is also known as black box testing, because it does not require any knowledge of the internal structure or code of the software.
- Functional testing involves testing the software from the user's perspective, by providing inputs and checking the outputs against the expected results.
- Functional testing can be performed at different levels of software development, such as unit testing, integration testing, system testing, and acceptance testing.
- Functional testing can be done manually or with the help of automated tools.
- Functional testing covers various aspects of software functionality, such as:
 - Validating the user interface and usability of the software
 - Checking the compatibility of the software with different platforms, browsers, devices, etc.
 - Verifying the business logic and workflows of the software
 - Testing the data processing and manipulation of the software
 - Evaluating the performance, reliability, and security of the software
- Functional testing can be classified into different types, such as:
 - Smoke testing: A preliminary testing to check the basic functionality and stability of the software
 - Sanity testing: A quick testing to verify the rationality and functionality of the software after minor changes or fixes
 - Regression testing: A testing to ensure that the existing functionality of the software is not affected by new changes or features
 - Exploratory testing: A testing to discover new defects or issues in

- the software by using creativity and intuition
- Usability testing: A testing to assess the user-friendliness and ease of use of the software
- Compatibility testing: A testing to check the interoperability and compatibility of the software with different environments and configurations
- Performance testing: A testing to measure the speed, scalability, and responsiveness of the software under different load conditions
- Security testing: A testing to identify and eliminate the vulnerabilities and risks of the software from unauthorized access or attacks
- Localization testing: A testing to ensure that the software can adapt to different languages, cultures, and regions
- User acceptance testing: A testing to validate the software by the end-users or stakeholders before the final release or deployment
- Functional testing has several benefits, such as:
 - It ensures that the software meets the user's expectations and requirements
 - It improves the quality and reliability of the software
 - It reduces the risk of failures and errors in the software
 - It enhances the customer satisfaction and confidence in the software
 - It facilitates the maintenance and support of the software

Test Data Suit Preparation software testing strategy

Test data suit preparation is the process of creating and organizing the data that is needed for testing the software application under test. Test data suit preparation is an important part of the test strategy document, which is a plan for defining the approach to the software testing life cycle.

Some of the steps involved in test data suit preparation are:

- **Analysis of data:** This step involves identifying the data requirements based on the test cases to be executed. The data should cover all the possible scenarios and inputs that the software application can handle. The data should also be realistic, valid, and consistent.
- **Data setup to mirror the production environment:** This step involves creating the test data in such a way that it resembles the real data that will be used in the production environment. This helps to ensure the reliability and accuracy of the test results. The test data can be generated using various methods, such as copying from the production database, using data masking techniques, using data generation tools, or creating synthetic data.
- **Determination of the test data clean-up:** This step involves deciding how to delete or restore the test data after the testing is completed. This helps to avoid data corruption, data leakage, or data duplication. The test data clean-up can be done manually, automatically, or using backup and restore mechanisms.

- **Test data management:** This step involves storing, maintaining, and updating the test data throughout the testing process. Test data management ensures that the test data is available, accessible, and secure for the testing teams. Test data management also involves tracking the changes, versions, and usage of the test data.

Test data suit preparation is a crucial aspect of software testing, as it affects the quality, efficiency, and effectiveness of the testing process. Test data suit preparation should be done carefully and systematically, following the best practices and guidelines of the test strategy document.

Alpha and Beta Testing of Products software testing strategy

- Alpha and beta testing are two types of user acceptance testing methodologies that help to build confidence in launching a product successfully.
- Both testing types rely on different goals, strategies, and processes.
- Alpha testing is focused on identifying bugs and validating that the software is functioning as a user would expect it to.
- Beta testing is a type of external user acceptance testing and is the final round of testing performed before a product is finally released to the wider audience.
- The purpose of beta testing is to test the user behavior, edge use cases, and compatibility with different platforms and also to spread product awareness.
- Alpha testing usually involves employees in the organization developing the software, while end users usually perform beta testing procedures.
- Alpha testing takes place in a controlled environment, while beta testing takes place at an end user's location or a client's facility.
- Alpha testing is conducted by internal employees of the organization using white box and black box practices.
- Beta testing involves a small set of real users who provide feedback on the usability, functionality, and quality of the product.
- Alpha testing is done before beta testing, and developers iron out any issues identified during alpha testing before releasing the beta version.
- Alpha and beta testing phases mainly focus on discovering the bugs from an already tested product and they give a clear picture of how the product is used by the real-time users.
- They also help in gaining experience with the product before its launch and valuable feedback is effectively implemented to increase the usability of the product.

Static Testing Strategies in Software Testing

Static testing is a software testing method that examines a program and its associated documents without executing the code. It is used to find and remove errors and ambiguities in the software development process at an early stage. It also helps to improve the quality and efficiency of software products.

There are two main types of static testing techniques:

- **Review:** This is a manual examination of the software artifacts, such as requirements, design, code, and test cases, to find defects and inconsistencies. Review can be done by the author of the artifact, by peers, by experts, or by customers. Review can be formal, informal, walkthrough, inspection, or audit, depending on the level of rigor and documentation required.
- **Static analysis:** This is an automated analysis of the software code using tools to detect defects, vulnerabilities, code smells, and other quality issues. Static analysis can be done by compilers, code analyzers, code metrics, code coverage, and code standards tools. Static analysis can help to enforce coding standards, improve code readability, optimize code performance, and prevent security breaches.

Formal Technical Reviews (Peer Reviews) Static testing strategy

- Formal technical reviews (FTRs) or peer reviews are a type of static testing strategy that involves a structured and systematic examination of software artifacts by a team of qualified reviewers.
- The main objectives of FTRs are to find defects, improve quality, verify compliance with standards and specifications, and facilitate knowledge transfer among the team members.
- FTRs can be applied to various software artifacts, such as requirements, design, code, test cases, user manuals, etc.
- FTRs follow a predefined process that consists of the following steps:
 - **Planning:** The review leader selects the review team, assigns roles and responsibilities, defines the scope and objectives of the review, and schedules the review meeting.
 - **Preparation:** The review team members study the software artifact to be reviewed and identify potential issues, questions, and comments. They also prepare a checklist of criteria to evaluate the artifact.
 - **Review meeting:** The review team meets to discuss the software artifact and share their findings and opinions. The review leader moderates the meeting and ensures that the review objectives are met and the review rules are followed. The review scribe records the issues and action items identified during the meeting.
 - **Rework:** The author of the software artifact corrects the defects and implements the action items reported by the review team. The author also provides evidence of the corrections and updates the artifact accordingly.
 - **Follow-up:** The review leader verifies that the rework has been done satisfactorily and that the software artifact meets the quality standards and specifications. The review leader also evaluates the effectiveness and efficiency of the review process and provides feedback to the review team and the management.

Walk Through (Walkthrough) Static testing strategy

- A walkthrough is a type of static testing technique that involves a review of a document or a code by a group of people, usually led by the author or the owner of the work product.
- The purpose of a walkthrough is to find defects, clarify issues, gather feedback, or achieve a common understanding of the work product among the participants.
- A walkthrough is usually informal and does not follow a strict process or a predefined checklist. The participants can ask questions, make comments, or suggest improvements during the walkthrough session.
- A walkthrough can be applied to any type of document or code, such as requirements, design, test cases, user manuals, etc. It can also be performed at any stage of the software development life cycle (SDLC).
- The typical steps of a walkthrough are:
 - The author or the owner of the work product prepares the document or the code and invites the participants for the walkthrough session. The participants can be peers, managers, customers, or other stakeholders who have an interest or a role in the work product.
 - The author or the owner of the work product presents the document or the code to the participants and explains the main objectives, assumptions, and logic behind it. The participants can ask questions, make comments, or suggest improvements during the presentation.
 - The author or the owner of the work product records the feedback, issues, and defects identified by the participants and decides how to address them. The author or the owner of the work product may also assign actions to the participants or other responsible parties to resolve the issues or defects.
 - The author or the owner of the work product updates the document or the code based on the feedback, issues, and defects and verifies that they have been resolved. The author or the owner of the work product may also conduct a follow-up walkthrough session to confirm that the work product meets the expectations of the participants.
- The advantages of a walkthrough are:
 - It is a simple and flexible technique that can be applied to any type of document or code and at any stage of the SDLC.
 - It helps to improve the quality of the work product by finding defects early and preventing them from propagating to the later stages of the SDLC.
 - It helps to enhance the communication and collaboration among the participants and to achieve a common understanding of the work product.

- It helps to reduce the cost and time of rework and testing by fixing the defects before they become more expensive and difficult to fix.
- The disadvantages of a walkthrough are:
 - It is a subjective and inconsistent technique that depends on the skills and experience of the participants and the author or the owner of the work product.
 - It may not cover all the aspects or scenarios of the work product and may miss some defects that are not obvious or visible during the presentation.
 - It may not follow a rigorous or a structured process and may not produce a formal or a detailed record of the feedback, issues, and defects.
 - It may consume a lot of time and resources of the participants and the author or the owner of the work product, especially if the work product is large or complex.

Code Inspection (Code Inspection) Static testing strategy

- Code inspection is a static testing technique that involves a systematic and formal review of the source code by a team of experts.
- The main objectives of code inspection are to find defects, improve code quality, ensure compliance with coding standards and guidelines, and facilitate knowledge transfer among developers.
- Code inspection can be performed at any stage of the software development life cycle, but it is usually done after coding and before testing.
- Code inspection follows a structured process that consists of the following steps:
 - Planning: The inspection leader selects the code to be inspected, assigns roles and responsibilities to the inspection team, and schedules the inspection meeting.
 - Preparation: The inspection team members study the code and identify potential defects, questions, and suggestions. They also prepare a checklist of items to be verified during the inspection.
 - Inspection meeting: The inspection team meets to discuss the code and review the findings. The inspection leader moderates the meeting and ensures that the inspection rules and objectives are followed. The inspection scribe records the defects and issues raised during the meeting.
 - Rework: The code author fixes the defects and issues identified during the inspection and verifies that the code meets the acceptance criteria.
 - Follow-up: The inspection leader checks that the rework has been done correctly and that no new defects have been introduced. The inspection leader also prepares a report that summarizes the inspection results and metrics.

- Code inspection has several benefits, such as:
 - Detecting defects early and reducing the cost of fixing them later.
 - Improving code quality and maintainability by enforcing coding standards and best practices.
 - Enhancing team collaboration and communication by sharing feedback and knowledge.
 - Increasing developer confidence and satisfaction by providing constructive criticism and recognition.
 - Supporting continuous improvement and learning by identifying areas for improvement and training needs.

Compliance with Design and Coding Standards (Coding Standards)

Static testing strategy

- Compliance with design and coding standards is a static testing technique that aims to ensure that the software product conforms to the predefined rules and guidelines for its structure, format, style, and quality.
- Compliance testing can be performed at different levels of software development, such as requirements, design, code, and documentation.
- Compliance testing can help to improve the readability, maintainability, portability, and security of the software product, as well as to prevent defects and errors.
- Compliance testing can be done manually or with the help of automated tools that can check the software artifacts against the standards and report any deviations or violations.
- Compliance testing can be based on internal or external standards, depending on the nature and scope of the software project. Some examples of internal standards are coding conventions, naming conventions, commenting guidelines, and documentation templates. Some examples of external standards are ISO/IEC 9126, IEEE 830, and MISRA C.
- Compliance testing can be integrated into the software development process as part of the quality assurance activities, such as reviews, inspections, audits, and walkthroughs. Compliance testing can also be performed as a separate activity before or after the software delivery.

Unit 5 - Software Maintenance and Software Project Management

- Software maintenance is the process of modifying and updating software after it has been delivered to the customer. Software maintenance can be classified into four types: corrective, adaptive, perfective, and preventive.
 - Corrective maintenance is the process of fixing errors or bugs that are discovered in the software after its delivery. Corrective maintenance can be reactive (responding to reported problems) or proactive (anticipating and preventing potential problems).
 - Adaptive maintenance is the process of modifying software to cope

with changes in the environment, such as new hardware, operating systems, or standards. Adaptive maintenance can be driven by customer requests or by technological evolution.

- Perfective maintenance is the process of enhancing software to improve its functionality, performance, usability, or quality. Perfective maintenance can be initiated by customer feedback, market analysis, or internal improvement goals.
- Preventive maintenance is the process of modifying software to reduce the complexity and improve the maintainability of the software. Preventive maintenance can involve activities such as code refactoring, documentation, testing, or reengineering.
- Software project management is the discipline of planning, organizing, leading, and controlling software projects. Software project management can be divided into five phases: initiation, planning, execution, monitoring and control, and closure.
 - Initiation is the phase where the project scope, objectives, feasibility, and stakeholders are defined. Initiation can involve activities such as project charter, business case, stakeholder analysis, or risk assessment.
 - Planning is the phase where the project activities, resources, schedule, budget, quality, and communication are planned. Planning can involve activities such as work breakdown structure, network diagram, Gantt chart, cost estimation, quality plan, or communication plan.
 - Execution is the phase where the project deliverables are produced according to the project plan. Execution can involve activities such as software development, testing, integration, or deployment.
 - Monitoring and control is the phase where the project progress, performance, quality, and risks are measured and compared with the project plan. Monitoring and control can involve activities such as status reports, variance analysis, change management, or quality control.
 - Closure is the phase where the project is formally completed and handed over to the customer. Closure can involve activities such as deliverable acceptance, project evaluation, lessons learned, or project archive.

Software as an Evolutionary Entity

- Software evolution is the continual development of a piece of software after its initial release to address changing stakeholder and/or market requirements.
- Software evolution is important because organizations invest large amounts of money in their software and are completely dependent on this software.
- Software changes are inevitable because there are many factors that change

during the life cycle of a piece of software, such as user needs, business goals, technology advances, security threats, etc.

- Software evolution can be fully manual (based on changes by software engineers), partially automated (e.g. using refactoring tools) or fully automated.
- Software evolution has been greatly impacted by the Internet, which enables faster and more frequent delivery of software updates, as well as more feedback from users and customers.
- Software evolution can be seen as a process of co-evolution with the software's domain, which includes the problem space, the solution space, and the application context .
- Software evolution can be guided by software planning, which can and should be based on process and system measures and models, obtained and maintained as a continuing process activity.
- Software evolution can be governed by software evolution laws, which are empirical observations that describe the patterns and trends of software change over time. Some examples of software evolution laws are:
 - Lehman's laws of software evolution, which state that software systems must be continually adapted or they become progressively less satisfactory, that the complexity of software systems increases unless work is done to maintain or reduce it, that the functionality of software systems tends to grow over time, etc.
 - Belady and Lehman's law of increasing change, which states that the number of changes per unit time in a software system increases as the system evolves.
 - Gall's law of software evolution, which states that software systems are best developed by starting with a simple system that works and then adding complexity incrementally.

Need for Maintenance and Maintenance Planning

- Maintenance is the process of keeping a system or equipment in a good working condition by performing regular inspections, repairs, replacements, or overhauls.
- Maintenance is necessary to ensure the reliability, availability, safety, and efficiency of a system or equipment.
- Maintenance can also improve the performance, quality, and productivity of a system or equipment, as well as reduce the operating costs, downtime, and environmental impacts.
- Maintenance planning is the process of determining the maintenance activities, resources, schedules, and procedures for a system or equipment.
- Maintenance planning is necessary to optimize the maintenance process and achieve the desired maintenance objectives and outcomes.

- Maintenance planning can also help to prevent or minimize failures, breakdowns, defects, or accidents, as well as improve the maintenance management and control.

Categories of Maintenance of Software

Software maintenance is the process of modifying and updating a software product after its delivery to correct faults, improve performance, or adapt to changing environments. Software maintenance can be classified into four categories, namely:

- **Corrective maintenance:** This type of maintenance involves fixing errors or bugs that are observed when the software is in use. These errors may be due to design flaws, coding mistakes, or hardware failures. Corrective maintenance aims to restore the software to its normal functioning state.
- **Preventive maintenance:** This type of maintenance involves anticipating and preventing potential problems or errors before they occur. Preventive maintenance aims to improve the quality, reliability, and maintainability of the software by applying techniques such as code refactoring, documentation, testing, and debugging. Preventive maintenance helps to reduce the cost and effort of future corrective maintenance.
- **Adaptive maintenance:** This type of maintenance involves modifying and updating the software to cope with changing requirements or environments. These changes may be due to new user needs, new technologies, new platforms, new standards, or new regulations. Adaptive maintenance aims to keep the software relevant and usable in the face of evolving situations.
- **Perfective maintenance:** This type of maintenance involves enhancing and improving the functionality, performance, usability, or design of the software. These enhancements may be due to user feedback, market demand, or competitive advantage. Perfective maintenance aims to increase the value and satisfaction of the software for the users and stakeholders.

These four categories of software maintenance are not mutually exclusive, and a software product may undergo one, two, or all types of maintenance throughout its lifespan. Software maintenance is an essential and ongoing activity that requires careful planning, management, and evaluation.

Preventive Maintenance (PM) of Software

- Preventive maintenance (PM) of software is the process of performing regular activities to prevent software defects, errors, failures, or degradation of performance or quality.

- PM of software aims to reduce the risk of software failures, improve user satisfaction, extend the software lifespan, and lower the maintenance costs.
- PM of software can include activities such as:
 - Updating the software to the latest version or patch to fix known bugs or security vulnerabilities.
 - Performing regular backups of the software data and configuration to ensure data integrity and recovery in case of disaster.
 - Checking the software logs and metrics to monitor the software health and performance, and identify any potential issues or anomalies.
 - Performing code reviews, testing, debugging, and refactoring to improve the software quality and maintainability.
 - Documenting the software design, architecture, features, and usage to facilitate the software understanding and communication among stakeholders.
 - Applying software engineering best practices and standards to ensure the software reliability, usability, scalability, and compatibility.
 - Educating and training the software users and developers to enhance their skills and knowledge of the software functionality and operation.
 - Soliciting and incorporating feedback from the software users and stakeholders to improve the software functionality and user experience.

Corrective Maintenance (CM) of Software

- Corrective maintenance (CM) of software is the process of fixing errors or defects in a software system after it has been delivered or deployed.
- CM can be triggered by user feedback, bug reports, testing results, or system monitoring.
- CM aims to restore the software system to its intended functionality and performance, and to ensure its reliability, security, and compatibility.
- CM can be classified into four types, depending on the nature and severity of the errors or defects:
 - Emergency maintenance: This type of CM is performed urgently to fix critical errors or defects that affect the normal operation of the software system or pose a serious risk to the users or the environment. Emergency maintenance requires immediate attention and action, and may involve temporary or partial solutions.
 - Adaptive maintenance: This type of CM is performed to adapt the software system to changes in the requirements, specifications, standards, or regulations. Adaptive maintenance may involve adding, modifying, or deleting features or functions of the software system to meet the new or revised expectations of the users or the stakeholders.

- Perfective maintenance: This type of CM is performed to improve the quality, performance, usability, or maintainability of the software system. Perfective maintenance may involve enhancing, optimizing, refactoring, or documenting the software system to increase its efficiency, effectiveness, or user satisfaction.
 - Preventive maintenance: This type of CM is performed to prevent or reduce the occurrence of errors or defects in the software system in the future. Preventive maintenance may involve correcting, updating, or upgrading the software system to eliminate potential sources of errors or defects, or to comply with the latest technologies or best practices.
- CM is an essential and inevitable part of the software development life cycle (SDLC), and it accounts for a significant portion of the software maintenance cost and effort.
 - CM can be performed using various methods, tools, and techniques, such as debugging, testing, patching, refactoring, version control, configuration management, or change management.
 - CM can be evaluated and measured using various metrics, such as the number, frequency, severity, or impact of errors or defects, the time, cost, or effort required to fix them, the quality, performance, or reliability of the software system after fixing them, or the user or stakeholder satisfaction with the software system after fixing them.

Perfective Maintenance (PM) of Software

- Perfective maintenance is the process of improving the functionality, performance, usability, or reliability of a software system without changing its intended purpose or behavior.
- Perfective maintenance is often performed in response to user feedback, changing requirements, new standards, or emerging technologies.
- Perfective maintenance can include activities such as adding new features, enhancing existing features, optimizing algorithms, refactoring code, updating documentation, or improving user interface design.
- Perfective maintenance can improve the quality and value of a software system, but it can also introduce new errors, increase complexity, or reduce maintainability if not done carefully and systematically.
- Perfective maintenance should be planned and prioritized based on the expected benefits and costs of each change, as well as the impact on the system's architecture, design, and testing.
- Perfective maintenance should follow the same software engineering principles and practices as any other software development activity, such as requirements analysis, design, coding, testing, debugging, and documentation.
- Perfective maintenance should be documented and tracked using appropriate tools and methods, such as version control, change management,

configuration management, and defect tracking.

Cost of Maintenance of Software

- Software maintenance is the process of modifying, updating, enhancing, or fixing software after its deployment to ensure its functionality, performance, security, and compatibility.
- Software maintenance can be classified into four types: corrective, adaptive, perfective, and preventive.
 - Corrective maintenance involves fixing errors or bugs that are detected in the software after its release.
 - Adaptive maintenance involves modifying the software to cope with changes in the environment, such as new hardware, operating systems, or regulations.
 - Perfective maintenance involves improving the software to meet new or changing user requirements, such as adding new features, enhancing usability, or optimizing performance.
 - Preventive maintenance involves modifying the software to prevent potential errors or problems from occurring in the future, such as refactoring, documenting, or testing the code.
- Software maintenance is an essential and ongoing activity that can consume a significant amount of time, effort, and resources. According to some studies, software maintenance can account for up to 90% of the total cost of ownership (TCO) of software over its entire life cycle.
- Software maintenance costs can vary depending on several factors, such as the type, size, complexity, quality, and age of the software, the number and frequency of changes, the availability and skills of the maintenance team, the tools and methods used, and the service level agreements (SLAs) with the customers or users.
- Software maintenance costs can be estimated using different methods, such as the percentage of development costs, the number of function points, the number of lines of code, the number of defects, or the number of change requests. However, these methods have their limitations and assumptions, and they may not reflect the actual costs accurately.
- Software maintenance costs can be reduced by applying best practices, such as following software engineering standards, using modular and maintainable design, applying quality assurance and testing techniques, using automation and configuration management tools, adopting agile and iterative development methods, and outsourcing or offshoring some maintenance tasks to external providers.

Software Re- Engineering (SR) of Software

- Software Re-Engineering is the process of examining and modifying an existing software system to improve its quality, performance, maintainability, and functionality .

- Software Re-Engineering involves a combination of sub-processes, such as reverse engineering, forward engineering, reconstructing, restructuring, etc.
- Reverse engineering is the process of extracting the design and specification of a software system from its source code.
- Forward engineering is the process of creating a new software system from the design and specification extracted by reverse engineering.
- Reconstructing is the process of transforming the source code of a software system into a more readable and understandable form.
- Restructuring is the process of improving the internal structure of a software system without changing its external behavior.
- Software Re-Engineering can be applied at different levels of abstraction, such as data, code, design, architecture, etc.
- Software Re-Engineering can have various benefits, such as reducing software cost, enhancing software quality, increasing customer satisfaction, and accelerating software delivery.
- Software Re-Engineering can also have some challenges, such as lack of documentation, complexity of legacy systems, resistance to change, and ethical issues.

Reverse Engineering (RE) of Software

- Reverse engineering of software is the process or method of analyzing an existing software product to understand its structure, functionality, and behavior.
- Reverse engineering of software can be used for various purposes, such as:
 - Developing new features or improving the existing ones by modifying the source code or the binary code of the software.
 - Debugging or fixing errors or vulnerabilities in the software by identifying the root causes and applying patches or updates.
 - Learning from the design or implementation of the software by studying its algorithms, data structures, or architectures.
 - Analyzing or comparing the software with other similar or competing products by measuring its performance, quality, or security.
 - Recovering the source code or documentation of the software if they are lost, damaged, or unavailable.
- Reverse engineering of software can be performed at different levels of abstraction, such as:
 - Machine code level: The lowest level of abstraction, where the software is represented as a sequence of binary instructions that are executed by the processor. This level requires tools such as disassemblers, decompilers, or debuggers to translate the machine code into a more readable form, such as assembly language or high-level language.
 - Intermediate code level: A higher level of abstraction, where the software is represented as a platform-independent code that is generated by a compiler or an interpreter. This level requires tools such as de-

compilers or deobfuscators to reconstruct the intermediate code into a high-level language.

- Source code level: The highest level of abstraction, where the software is represented as a human-readable code that is written in a programming language. This level requires tools such as source code analyzers, parsers, or refactoring tools to understand and manipulate the source code.
- Reverse engineering of software can be challenging and complex, depending on the size, complexity, and protection of the software. Some of the common challenges or difficulties are:
 - The software may be obfuscated, encrypted, or compressed to make it harder to reverse engineer.
 - The software may use anti-reversing techniques, such as code self-modification, code injection, or anti-debugging to prevent or detect reverse engineering attempts.
 - The software may have incomplete, inaccurate, or misleading documentation or comments to confuse or mislead the reverse engineer.
 - The software may have proprietary, undocumented, or complex algorithms, data structures, or architectures that are difficult to understand or replicate.
 - The software may have legal, ethical, or contractual restrictions that prohibit or limit reverse engineering.

Software Configuration Management Activities

Software Configuration Management (SCM) is the process of identifying, controlling, and tracking the changes and versions of software artifacts. SCM aims to ensure the integrity, consistency, and quality of software products throughout the software development life cycle. SCM involves the following activities:

- **Configuration identification:** This activity involves defining and naming the software items and components that make up the software product, and establishing the relationships and dependencies among them. Configuration identification also involves defining the baselines, which are the approved versions of software items that serve as the basis for further development or delivery.
- **Configuration control:** This activity involves managing and controlling the changes and modifications to the software items and components, and ensuring that they are consistent with the requirements and specifications. Configuration control also involves evaluating and approving the change requests, implementing and testing the changes, and updating the baselines and documentation accordingly.
- **Configuration status accounting:** This activity involves recording and reporting the status and history of the software items and components, and the changes and modifications made to them. Configuration status accounting also involves tracking and auditing the configuration items

and baselines, and providing visibility and traceability of the software configuration throughout the software development life cycle.

- **Configuration audit:** This activity involves verifying and validating that the software items and components conform to the requirements and specifications, and that the changes and modifications are properly implemented and documented. Configuration audit also involves checking and reviewing the compliance and quality of the software configuration, and identifying and resolving any discrepancies or issues.
- **Configuration management planning:** This activity involves defining and documenting the policies, procedures, standards, tools, and roles and responsibilities for performing the SCM activities. Configuration management planning also involves establishing and maintaining the SCM environment, which includes the hardware, software, and network resources required for SCM.

Change Control Process in Software Project Management

Change control is the process of managing and assessing changes to a software project and its procedures. Change control can help a project manager to regulate the project and alter it based on changing environments, conditions or requirements. Change control can also help to avoid unnecessary changes that might disrupt the software development or delivery.

The change control process in software project management typically involves the following steps:

- **Change request initiation:** In this step, a change request is submitted by anyone on the project team, a stakeholder, a customer, or a user. The change request should describe the change, the reason for the change, the expected benefits and costs of the change, and the priority and urgency of the change.
- **Change request assessment:** In this step, the project manager reviews the change request and determines whether it is valid, feasible, and aligned with the project scope, objectives, and constraints. The project manager may consult with the project team, the stakeholders, the customers, or the users to gather more information or feedback about the change request.
- **Change request analysis:** In this step, the project manager analyzes the impact of the change request on the project schedule, budget, quality, risk, and scope. The project manager may use various tools and techniques, such as impact analysis, cost-benefit analysis, risk analysis, or trade-off analysis, to evaluate the pros and cons of the change request and to identify any dependencies, assumptions, or alternatives.
- **Change request implementation:** In this step, the project manager presents the change request and the analysis results to the change control board (CCB), which is a group of authorized people who have the power to approve or reject the change request. The CCB may approve the change request as it is, approve it with modifications, defer it for further

review, or reject it. If the change request is approved, the project manager updates the project plan, the project baseline, and the project documents accordingly and communicates the change to the project team and the relevant stakeholders.

- **Change request evaluation:** In this step, the project manager monitors and controls the implementation of the change request and verifies that the change has been completed successfully and has met the expected outcomes. The project manager may use various tools and techniques, such as quality control, testing, verification, or validation, to ensure that the change has not introduced any errors, defects, or risks to the software product or the project. The project manager may also collect feedback from the project team, the stakeholders, the customers, or the users to measure the satisfaction and the value of the change.

The change control process in software project management is an important process to help manage large and complex software projects with multiple moving parts. By following the change control process, the project manager can ensure that the changes are effectively defined, reviewed, and approved before implementation and that the changes are smoothly integrated into the software product and the project.

Software Version Control in software project management

- Software version control (SVC) is a management strategy to track and store changes to a software development document or set of files that follow the development project from beginning to end-of-life .
- SVC is a component of software configuration management, which is the process of controlling the evolution of a software system.
- SVC helps software teams to:
 - Manage changes to source code over time and avoid conflicts or errors .
 - Collaborate effectively and coordinate their work on a shared codebase .
 - Maintain the history and traceability of the software development process .
 - Recover previous versions of the code in case of bugs, errors, or failures .
 - Experiment with new features or ideas without affecting the main codebase .
- SVC systems are software tools that help implement SVC by providing features such as:
 - Revision identification: a number or letter code that indicates the version of the code.
 - Branching: creating a copy of the code for parallel development .
 - Merging: combining changes from different branches into a single version .

- Tagging: labeling a specific version of the code for reference or release .
- Committing: saving changes to the code and adding a message to describe them .
- Pushing and pulling: transferring changes between local and remote repositories .
- SVC systems can be classified into two main types: centralized and distributed .
 - Centralized SVC systems have a single server that stores the master copy of the code and manages the access and permissions of the developers . Examples of centralized SVC systems are Subversion (SVN) and Team Foundation Version Control (TFVC) .
 - Distributed SVC systems have multiple repositories that can act as servers or clients, allowing developers to work offline and synchronize their changes later . Examples of distributed SVC systems are Git and Mercurial .
- SVC is an essential practice for software project management, as it enables quality assurance, testing, deployment, and maintenance of the software product . SVC also facilitates communication, collaboration, and transparency among the software team and stakeholders .

An Overview of CASE Tools in Software Project Management

- CASE tools are a set of software application programs, which are used to automate SDLC activities.
- CASE stands for Computer-Aided Software Engineering.
- CASE tools are used by software project managers, analysts and engineers to develop software system of high quality and free of defects.
- CASE tools can be classified into three categories based on the stage of software development life cycle they support:
 - Upper CASE tools: These tools support the early stages of software development, such as analysis, design and specification. Examples are diagram tools, process modeling tools, data modeling tools, etc.
 - Lower CASE tools: These tools support the later stages of software development, such as implementation, testing and maintenance. Examples are code generators, debuggers, testing tools, configuration management tools, etc.
 - Integrated CASE tools: These tools support the entire software development life cycle, from analysis to maintenance. They provide a seamless integration of upper and lower CASE tools, and facilitate communication and coordination among different stakeholders. Examples are Appian, Pega, Kissflow, etc.
- CASE tools have several advantages, such as :
 - Improving the quality and consistency of software products by enforcing standards and guidelines.
 - Reducing the cost and time of software development by automating

repetitive and tedious tasks.

- Enhancing the productivity and creativity of software developers by providing them with graphical and interactive tools.
- Facilitating the communication and collaboration among different stakeholders by providing a common platform and documentation.
- Supporting the reuse and maintenance of software components by providing a repository and version control system.
- CASE tools also have some disadvantages, such as:
 - Requiring a high initial investment and training cost to acquire and use the tools.
 - Creating a dependency on the vendor and the tool for software development and maintenance.
 - Introducing new errors and complexities due to the limitations and bugs of the tools.
 - Restricting the flexibility and innovation of software developers by imposing predefined methods and techniques.

Estimation of Various Parameters such as Cost and Time in software project management

- Estimation is the process of predicting the required resources, effort, cost and duration of a software project based on available information and assumptions.
- Estimation is essential for planning, controlling and managing software projects, as well as for communicating with stakeholders and setting realistic expectations.
- Estimation can be done at different levels of detail and accuracy, depending on the stage of the project and the availability of information.
- Estimation can be done using various techniques, such as expert judgment, analogy, decomposition, parametric, or hybrid methods.
- Parametric estimation is one of the most widely used and reliable techniques for software project estimation. It involves using mathematical models, formulas, or algorithms to calculate the project parameters based on historical data and statistical relationships.
- Parametric estimation requires identifying the key variables or parameters that affect the project, such as size, complexity, quality, productivity, etc. These variables can be measured using different units, such as lines of code, function points, use cases, etc.
- Parametric estimation also requires collecting and analyzing historical data from similar projects or industry benchmarks to establish the values and ranges of the variables, as well as the coefficients and constants of the formulas or algorithms.
- Parametric estimation can be applied at different levels of granularity, such as project, phase, activity, or task level. It can also be adjusted or refined as the project progresses and more information becomes available.
- Parametric estimation can provide accurate and consistent results, as well

as facilitate sensitivity analysis, risk analysis, and trade-off analysis. However, it also has some limitations, such as the need for reliable and relevant data, the validity and applicability of the models or formulas, and the uncertainty and variability of the variables and parameters.

- Some examples of parametric estimation models or methods for software projects are COCOMO, Function Point Analysis, Use Case Points, and Story Points.

Efforts to Improve Software Quality in Software Project Management

Software quality is the degree to which a software product meets the requirements and expectations of its stakeholders. Software quality can be measured by various attributes, such as functionality, reliability, usability, efficiency, maintainability, and portability.

Software project management is the process of planning, organizing, leading, and controlling software projects. Software project management involves various activities, such as defining the scope, schedule, budget, and quality goals of the project, managing the risks, resources, and stakeholders, monitoring and controlling the progress and performance, and delivering the software product to the customer.

Efforts to improve software quality in software project management can be classified into three categories: prevention, appraisal, and correction.

- Prevention efforts are aimed at avoiding or reducing the occurrence of defects in the software product. Prevention efforts include applying software engineering principles and standards, conducting reviews and inspections, performing verification and validation, using quality assurance techniques, and implementing quality management systems.
- Appraisal efforts are aimed at detecting and measuring the defects in the software product. Appraisal efforts include testing, auditing, and evaluating the software product against the quality criteria and specifications.
- Correction efforts are aimed at removing or fixing the defects in the software product. Correction efforts include debugging, reworking, and maintaining the software product.

Some of the benefits of improving software quality in software project management are:

- Increased customer satisfaction and loyalty
- Reduced costs and risks of rework, failure, and litigation
- Enhanced reputation and competitiveness of the software organization
- Improved productivity and efficiency of the software development process
- Facilitated compliance with regulatory and contractual requirements

Schedule/Duration of Maintenance in software project management

- Software maintenance is the process of modifying and updating a software system after its delivery to correct faults, improve performance, adapt to changing environments, or add new features.
- Software maintenance can be classified into four types: corrective, adaptive, perfective, and preventive maintenance.
- The schedule or duration of software maintenance depends on various factors, such as the size and complexity of the software system, the quality and reliability of the software, the availability and skills of the maintenance staff, the user requirements and feedback, the budget and resources allocated for maintenance, and the maintenance policies and standards adopted by the organization.
- Some of the challenges or issues that affect the schedule or duration of software maintenance are:
 - The difficulty of estimating the effort and time required for maintenance tasks, especially for adaptive and perfective maintenance, which may involve significant changes to the software system.
 - The uncertainty and variability of user requests and expectations, which may change over time or differ from the original specifications of the software system.
 - The dependency and compatibility of the software system with other software or hardware components, which may require coordination and synchronization of maintenance activities across different systems or platforms.
 - The evolution and obsolescence of the software system, which may require frequent updates or upgrades to keep up with the changing technologies and standards.
 - The availability and retention of the maintenance staff, who may have limited knowledge or experience of the software system, or may leave the organization for various reasons.
 - The documentation and communication of the maintenance activities, which may be incomplete, inconsistent, or outdated, leading to confusion or errors in the maintenance process.
- Some of the best practices or guidelines for planning and managing the schedule or duration of software maintenance are:
 - Establishing a clear and realistic scope and objectives of the maintenance project, based on the user needs and priorities, and the organizational goals and constraints.
 - Applying appropriate methods and tools for estimating, tracking, and controlling the maintenance effort and time, such as function point analysis, COCOMO, Gantt charts, or earned value management.
 - Adopting a systematic and structured approach for performing and documenting the maintenance activities, such as the software maintenance life cycle model, which consists of six phases: identification, analysis, design, implementation, testing, and delivery.

- Implementing quality assurance and testing techniques to ensure the correctness and reliability of the maintenance work, such as code reviews, inspections, testing, or debugging.
- Developing and maintaining a comprehensive and consistent documentation of the software system and the maintenance activities, such as the software configuration management, which involves the identification, control, audit, and status reporting of the software components and their changes.
- Communicating and collaborating effectively with the stakeholders involved in the maintenance project, such as the users, developers, managers, or vendors, to ensure the alignment and satisfaction of their expectations and requirements.

Constructive Cost Models (COCOMO) in software project management

- Constructive Cost Models (COCOMO) are a set of algorithms and equations that estimate the effort, cost and schedule of software projects based on various inputs and assumptions.
- COCOMO was developed by Barry Boehm in 1981 and has been revised and updated over the years to reflect the changes in software engineering practices and technologies.
- COCOMO consists of three levels of models: basic, intermediate and detailed, each with increasing accuracy and complexity.
- The basic COCOMO model estimates the effort and duration of a software project based on the size of the software product measured in lines of code (LOC) or function points (FP) and a mode of development (organic, semi-detached or embedded).
- The intermediate COCOMO model refines the basic model by introducing a set of cost drivers that account for various factors that affect the productivity and quality of software development, such as personnel capability, experience, complexity, reliability, etc.
- The detailed COCOMO model further refines the intermediate model by dividing the software product into different components or modules and applying different cost drivers and effort multipliers to each component based on its characteristics and dependencies.
- COCOMO can be used for planning, budgeting, monitoring and controlling software projects, as well as for comparing and evaluating alternative software development methods and tools.

Resource Allocation Models (RAIM) in software project management

Resource allocation models (RAIM) are methods or tools that help project managers plan, schedule, and assign resources to project tasks in order to achieve project objectives. Resources can be material, equipment, financial, or human resources. Resource allocation models can help project managers balance the

three constraints of time, scope, and cost, as well as optimize the utilization and productivity of resources.

Some of the common resource allocation models in software project management are:

- **Critical path method (CPM):** This model identifies the longest sequence of tasks that determines the project duration, and allocates resources to the critical tasks that cannot be delayed without affecting the project completion date. CPM can help project managers avoid resource overallocation, reduce project risks, and improve project quality.
- **Agile resource allocation:** This model follows the agile principles of software development, which emphasize flexibility, collaboration, and customer satisfaction. Agile resource allocation involves allocating resources to short iterations or sprints, where the project scope and requirements are constantly refined and prioritized. Agile resource allocation can help project managers adapt to changing needs, deliver value faster, and increase customer feedback.
- **Resource leveling:** This model aims to minimize the fluctuations or variations in resource usage throughout the project. Resource leveling involves adjusting the start and end dates of tasks, or splitting tasks into smaller units, to ensure that the same amount of resources are used in each time period. Resource leveling can help project managers avoid resource shortages, reduce resource costs, and improve resource efficiency.

Resource allocation models can vary depending on the project size, complexity, and methodology. Project managers should choose the model that best suits their project needs and goals, and use appropriate tools and techniques to implement and monitor the resource allocation process.

Software Risk Analysis and Management in software project management

- Software risk is the probability of occurrence for uncertain events and their potential for loss within an organization.
- Software risk analysis is the process of identifying, categorizing, and estimating the level, likelihood, and impact of software risks .
- Software risk management is the process of planning, implementing, monitoring, and controlling activities to minimize the negative effects and maximize the positive opportunities of software risks.
- Software risk analysis and management is an important component of software project management, as it helps to ensure the quality, reliability, security, and performance of software products and services .
- Software risk analysis and management involves the following steps :
 - Risk identification: The process of finding and documenting the sources and types of software risks, such as technical, operational, organizational, legal, or environmental risks.

- Risk analysis: The process of assessing the probability and severity of software risks, using qualitative or quantitative methods, such as risk matrices, risk models, or risk metrics.
- Risk prioritization: The process of ranking software risks according to their importance and urgency, using criteria such as risk exposure, risk tolerance, or risk appetite.
- Risk mitigation: The process of developing and implementing strategies to reduce or eliminate software risks, such as risk avoidance, risk transfer, risk reduction, or risk acceptance.
- Risk monitoring: The process of tracking and measuring the status and performance of software risks and their mitigation actions, using tools such as risk registers, risk dashboards, or risk reports.
- Risk control: The process of reviewing and adjusting the software risk analysis and management plan, based on the feedback and changes from the risk monitoring process, using techniques such as risk audits, risk reviews, or risk updates.

Software Project Management

- Software project management is the process of planning and leading software projects.
- Software projects are projects that involve the creation, modification, or maintenance of software products or systems.
- Software project management is a sub-discipline of project management, which is the application of knowledge, skills, tools, and techniques to achieve specific goals and meet specific success criteria.
- Software project management involves variables such as time, quality, cost, scope, risk, and stakeholder satisfaction.
- Software project management typically follows a life cycle model, which defines the phases and activities of a project from initiation to closure.
- Some common software project life cycle models are waterfall, agile, iterative, incremental, spiral, and V-model.
- Software project management best practices include defining clear and realistic objectives, establishing a project plan, managing risks and changes, communicating effectively, monitoring and controlling progress, and ensuring quality and customer satisfaction.
- Software project management also requires the use of software tools and techniques, such as project management software, software estimation methods, software metrics, software testing, software configuration management, and software documentation .
- Software project management is a challenging and dynamic field that requires constant learning and adaptation to changing requirements, technologies, and environments.